1. Titel

Overvågning og datalogning af Systemair Danvent DV10 ventilationssystem

MVP fra sensor til dashboard



04/12/2025

Indhold

1. Titel 1
Indhold 2
GitHub link til anvendte koder og konfigurationer: 3
2. Introduktion 4
Problemformulering 4
Formål 4
3. Teoretisk fundament 5
Projektet overordnet. 5
Elektriske principper og fænomener 5
RS-485 6
Modbus RTU 6
Kodesprog og struktur 7
Netværkstopologi 7
Komponentliste: 7
Hardware 8
Software 8
Protokoller og standarder 8
Kravspecifikation for MVP: 8
4. Metodologi 10
Systemtopologi 10
Use case 11
Klassediagram 12
Klasser/funktioner i main.py 13
Protokoller der er blevet brugt 14
Industrielle protokoller (IEC, ISO, Sparkplug, osv.) 14
Industrielle Standarder 14
Metode afgrænsning: 19
5. Resultater og analyse 19
Præsentation af prototype 19
Systemoversigt: 19
Edge-laget. 19
MQTT 21
Ingestor og Data-transport 22
API-laget 22
Logging-system 24
Frontend-visualisering 24
Dataintegration 25
Testprotokol 26
Funktionalitet af prototypen 26
6. Diskussion 30
Praktisk anvendelighed 30
Opsummeret: 30

Udfordringer og muligheder 31
Udfording: Single point of failure. 31
Muligheder inden for kort tidshorisont. 31
7. Perspektivering 31
Identifikation af udviklingsmuligheder 31
Standardisering og skalering 31
Infrastruktur og driftsikkerhed 31
Edge-enheden og OTA 32
Datakvalitet og robusthed 32
Udvidelse af webserver funktionalitet 32
Fremtidige udfordringer og løsninger 32
8. Referencer 34
Bilag 34
Bilag 1 – Udvidet komponentliste 34
Hardware 34
Bilag 2: 36
Bilag 3: 36
Bilag 4 38
Bilag 5 40
Bilag 6: 49

GitHub link til anvendte koder og konfigurationer:

https://github.com/Amir804-cell/iot-monitoring_V2

2. Introduktion

Problemformulering

En kunde ønsker at sætte overvågning på et ventilationssystem, der ikke selv kan logge sine data. Kunden har selv domæneviden og kan optimere systemet, men mangler indsigt i systemets opførsel over tid.
Hvordan kan man bygge et system, som giver kunden løbende adgang til data, aktuelle og historiske, som kan bruges til at sikre ventilationssystemets funktionalitet over tid?

Formål

Projektet har til formål at udvikle en løsning, som kan opsamle data fra et ventilationssystem uden indbygget datalogning.

Kunden ønsker indsigt i systemets opførsel over tid via sikker dataopsamling, lagring og visualisering.

Målet er dermed at sende data til en backend, logge hensigtsmæssige data, og præsentere data i dashboards. Et dashboard bliver tiltænkt udviklerteamet og et andet på en lokal webserver bliver tiltænkt kundens personale med ventilationsdriftskompetencer.

Overførsel af data sker, groft skitseret, gennem denne pipeline:

Ventilationsanlæg - Edge-enhed - Database - API - Dashboard.

De tre grundtrin i projektet kan dermed oplistes således:

1. Interface med ventilationsanlægget og frembringe en løsning, som kan styre anlægget og lave meningsfulde dataudtræk igennem Edge-enheden.
2. Byg en backend som kan modtage data fra Edge-enheden, bestående af en tidsserie database til datalagring, en API til datahåndtering og et dashboard til datavisualisering.
3. I projektets sidste fase implementeres udviklingsteamets end-to-end integration, hvor reelle data fra anlæggets sensorer bliver lagret og visualiseret med en MVP (Minimum Viable Product) løsning.

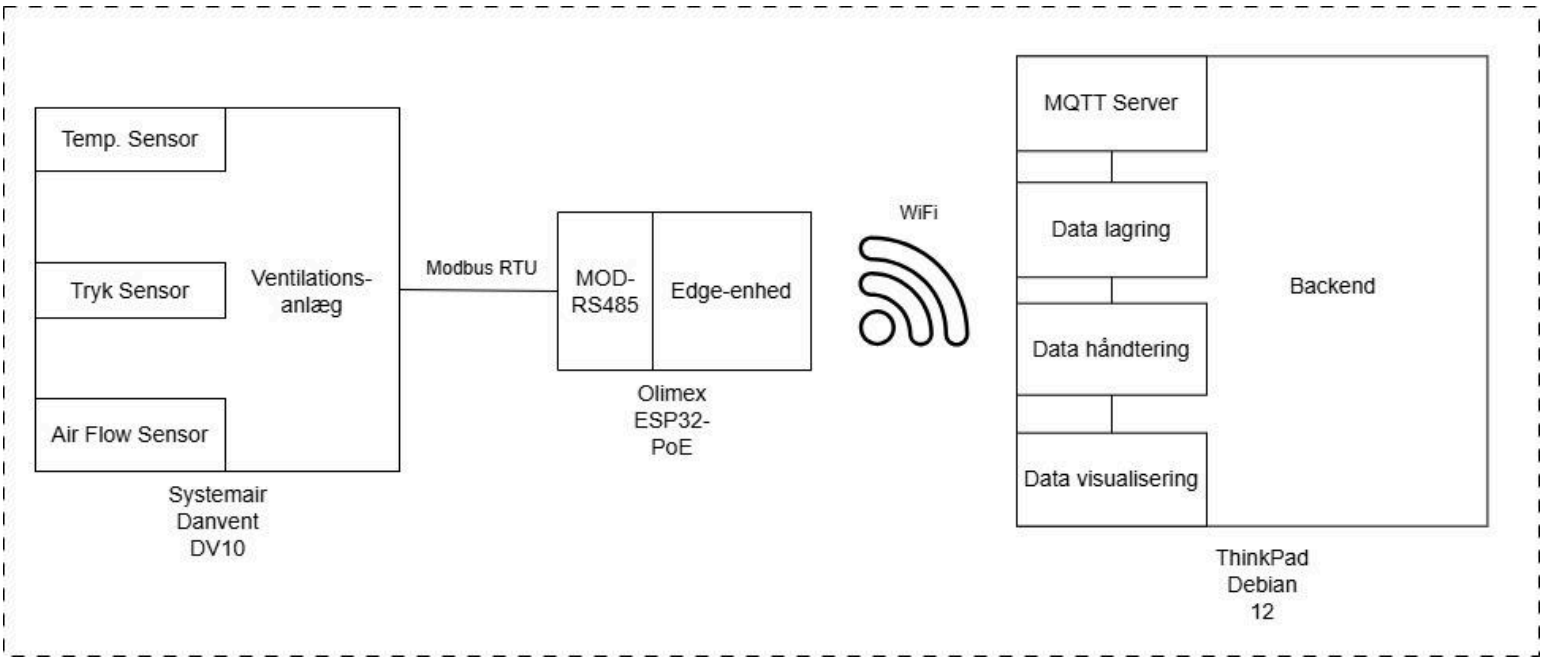
3. Teoretisk fundament

Projektet overordnet.

Systemet er opbygget af tre hovedkomponenter:

- Ventilationsanlæg (Systemair Danvent DV10)
- Edge-enhed (Olimex ESP32-PoE med MOD-RS485)
- Back-end (ThinkPad T470s med Debian 12)

Hver hovedkomponent, og deres projektrelevante delkomponenter, er illustreret forneden i blokdiagram 1.



Elektriske principper og fænomener

Kommunikation mellem DV10 anlægget og Olimex ESP32-POE etableres ved sidstenævntes tilslutning til, anlæggets digitale kontroller, Corrigo E[1].

[1] https://www.regincontrols.com/globalassets/pimmediafiles/documents/corrigo_e_3.2_prsh_en.pdf

Corrigo E's Modbus A/B-ledere tilsluttes blot til tilsvarende terminaler på MOD-RS485-modulet. Forbindelsen anvender differential signalering, hvilket sikrer robust dataoverførsel i miljøer med elektromagnetisk støj.[2]

RS-485

RS-485 fungerer som det fysiske lag i løsningen. [3] For at muliggøre dataudtræk anvendes Modbus RTU på datalink-laget.[4] Trafikken går over Olimex ESP32-POE enhedens UART RX og TX Pins.

Modbus RTU

Protokollen opererer efter en master/slave-struktur, hvor Olimex ESP32-POE fungerer som master og forespørger Corrigo E om værdier fra definerede Modbus registre. De indhentede data videresendes til back-end via API, lagres i en tidsseriedatabase og visualiseres i dashboards.

NB vedrørende registeradresser

Ved aflæsning af registre skal der tages højde for forskellen mellem Modbus-adressering og udviklingsmiljøets adressering. Modbus-adresser starter ved 1, mens adresser i udviklingsmiljøet starter ved 0.

Eksempel: '*Outdoor Temp'* er placeret ved Corrigo E's Input Register 1, men skal angives som Input Register 0 i udviklingsmiljøet.

NB vedrørende Modbus RTU rå værdier

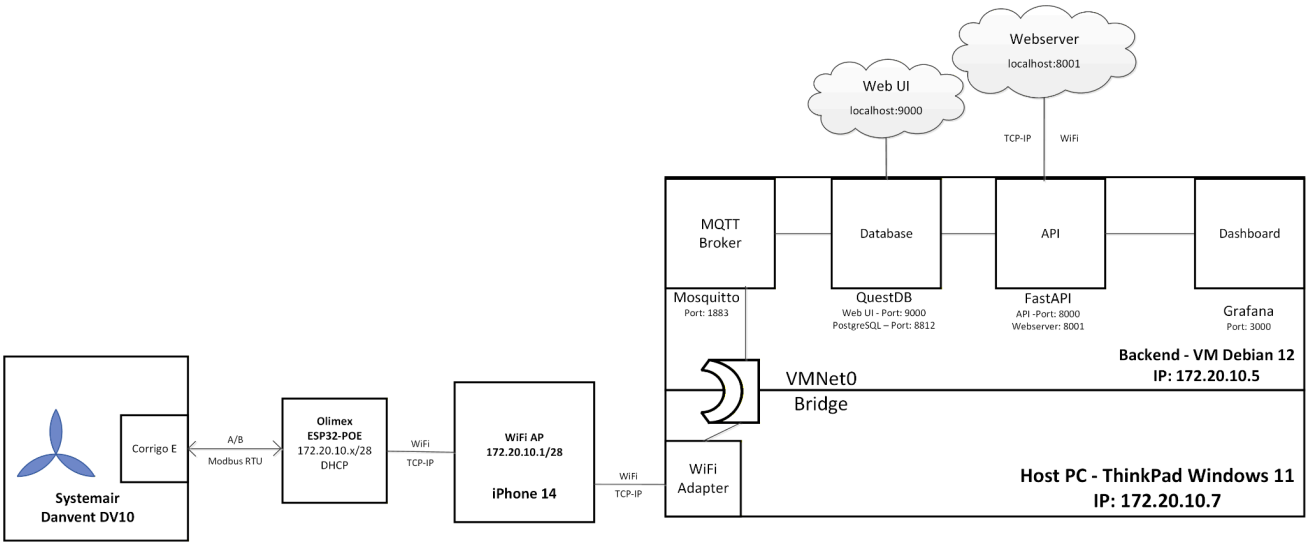
Modbus bruger registrer til at skrive eller læse, som hver består af 16 bit data. 16 bit (65536 eller unikke data punkter) understøtter ikke decimal punkter og kan derfor ikke angive en værdi som fx 19.9 c. Løsningen er at sende hele tal, en integer, som indeholder den samlede data (199) og som kan omregnes til værdi med decimal tal, altså en float, fx. 199/10 = 19.9.

På denne måde er data for temperatur, tryk, flow og tid udregnet.

[2] https://www.cdebyte.com/news/606
[3] https://en.wikipedia.org/wiki/RS-485
[4] https://www.researchgate.net/figure/Modbus-protocol-in-ISO-OSI-model_fig1_326109596

Kodesprog og struktur

Netværkstopologi



Systemet er bygget op som en simpel og funktionel stjernetopologi hvor alle noder er centreret om en Debian 12 server som fungerer som MQTT Broker, databaseserver og API-host.

Debian-serverens services:
– Mosquitto (port 1883) modtager alle MQTT-publikationer.
– QuestDB fungerer som tidsseriedatabase med PostgreSQL ingestion (port 8812) og tilbyder et web-UI (port 9000).
– FastAPI-serveren (port 8000), stiller data til rådighed for systemets øvrige lag og kører en webserver (port 8001).
– Grafana (port 3000), som fungerer som visualiserings- og dashboardoverflade.

På Edge siden kommunikerer DV10 anlæggets Corrigo E kontroller med Edge-enheden via Modbus RTU over et A/B differential signalpar. Som Modbus Master forespørger Edge-enheden Modbus Slave efter værdier på specifikke registeradresser og sender dataen videre til MQTT Brokeren på Debian Serveren. Herfra kan data opbevares i database-laget og forvaltes gennem API-laget.

Serveren fungerer som central hub og er det primære single point of failure i arkitekturen, hvilket er forventeligt i en MVP-opsætning.

Komponentliste:

NB: Den samlede komponentlisten er renskrevet med AI, se bilag 6, Prompt A.

Hardware

- **Edge-enhed:** Olimex ESP32-POE med MOD-RS485 modul til Modbus RTU-kommunikation.
- **Ventilationsanlæg:** Systemair Danvent DV10 med Corrigo E kontroller (Modbus RTU slave).
- **Backend-platform:** Lenovo ThinkPad T490 med VM Debian 12 backend.
- **Netværksudstyr:** iPhone 14 (WiFi Access Point - DHCP, subnet 172.20.10.0/28).

Software

- **MQTT-broker:** Eclipse Mosquitto.
- **Database:** QuestDB.
- **API-lag:** FastAPI.
- **Dashboard:** Grafana.
- **Operativsystemer:** Debian 12 (backend), Windows 11 (host-system).

Protokoller og standarder

- **Modbus RTU** (RS485 differential bus mellem Corrigo E og ESP32-POE).
- **MQTT** (data publikationer fra edge-enheden).
- **HTTP/REST** (API-udstilling via FastAPI).
- **TCP/IP** (kommunikationslag i hele IP-topologien).
- **JSON** (datastrukturering mellem edge og backend).

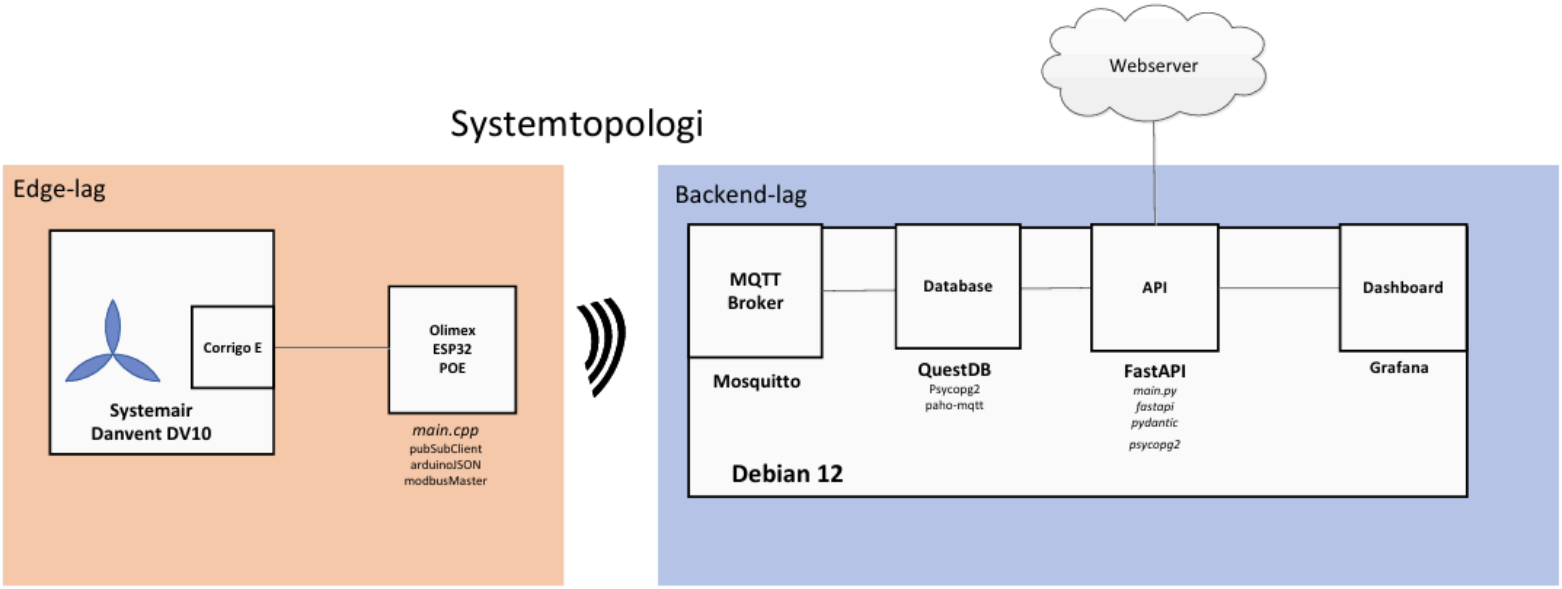
Kravspecifikation for MVP:

1. Edge enheden kan lave dataudtræk fra alle relevante Modbus-registre
2. Edge enheden kan konvertere Modbus råværdi værdier for teknisk personale.
3. Edge enheden angiver værdier som stemmer med værdierne vist i DV10 anlæggets reference-enhed, inden for 10% tolerance.
4. MQTT broker acceptere forbindelser fra Edge enhed og backend, og udfører dataoverførsel fra MQTT publisher på Edge enheden, til MQTT subscriber på backend, med latency under 500ms.
5. QuestDB indeholder mindst én tabel til ventilationsdata med timestamp.

6. QuestDB lagrer data som tidsseriedata fra ventilationsanlægget.
7. FastAPI eksponerer endpoints, som henter data fra QuestDB.
8. Systemet leverer logs i SPDlogs-format vedrørende MQTT Broker.
9. Systemet leverer logs i SPDlogs-format vedrørende API-lag webserver.
10. Grafana dashboard visualiserer seks datapunkter fra ventilationsanlægget.
11. Webserver på API-laget kan fremvise live datasæt med runmode, runtime, temperatur, tryk og flow fra DV10 anlægget.
12. MQTT-payload er JSON-formateret og bruger topic-struktur og meddelelsestyper formateret efter Sparkplug B's principper. (NBIRTH, DBIRTH).
13. Unit tests skal være defineret for Edge-enheden samt for systemets backend komponenter: MQTT-broker, QuestDB, FastAPI og Grafana.
14. Alle implementerede unit tests skal bestås.

4. Metodologi

Systemtopologi



Edge-lag

Det fysiske lag omfatter en Systemair Davent DV10 med Corrigo-E kontroller, et MOD-RS485 interface-modul samt en Olimex ESP32-POE. DV10-anlægget leverer driftsdata via Corrigo-E kontrolleren, hvor Modbus RTU over RS485 anvendes som kommunikationsprotokol. Olimex-enheden modtager de rå Modbus-registre, foretager skalering til procesværdier (°C, %, Pa, m³/h) og publicerer normaliserede data som MQTT-beskeder i Sparkplug-B format via Wi-Fi. Edge-enheden er alene tildelt ansvar for datakonvertering, idet registerlayout og skalafaktorer kun er kendt på dette lag. Andre værdier som 'runmode' kræver ingen omregning.

Backend-lag

Backend-laget er installeret på en Debian 12 virtuel maskine. Her modtager en Mosquitto-broker de publicerede MQTT-beskeder, hvorefter en Subscriber-proces indsætter datapunkter i QuestDB. FastAPI eksponerer derefter data via REST-endpoints, som anvendes af Grafana til visning af systemets driftsstatus. Adgangsrettigheder er begrænset til udviklingsholdet i henhold til "principle of least privilege"[5].

[5] https://en.wikipedia.org/wiki/Principle_of_least_privilege

Brugergrænseflade

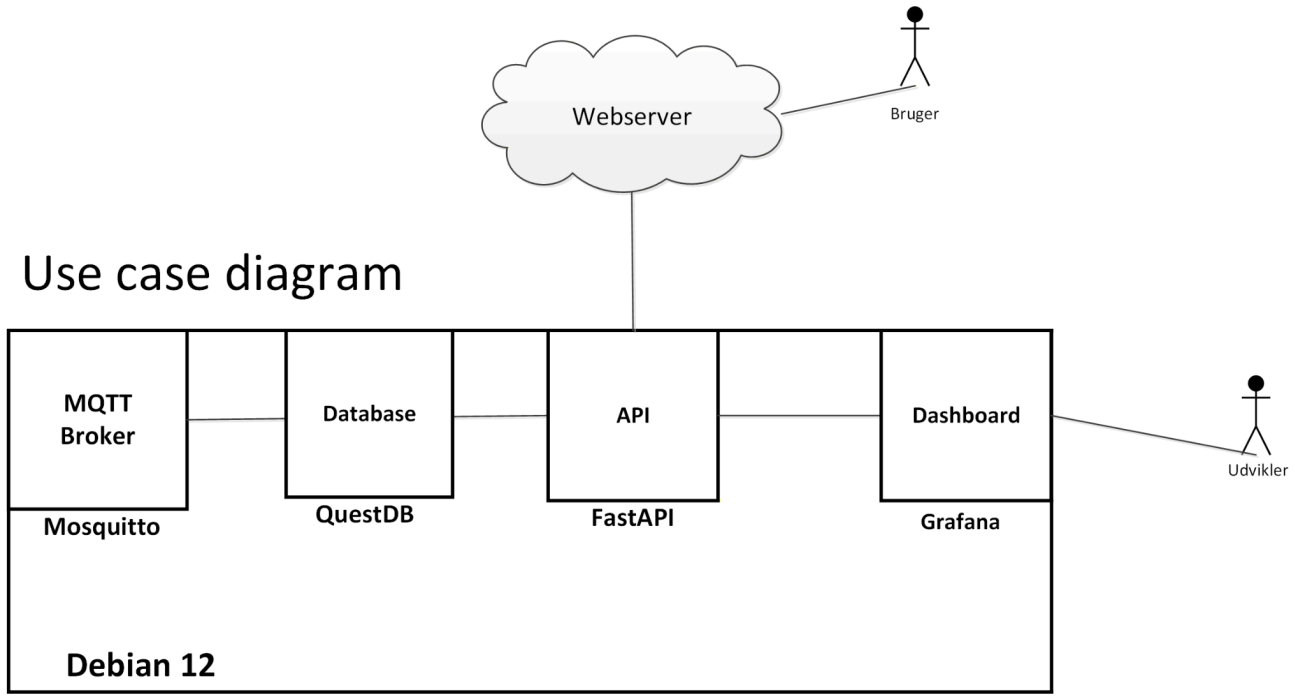
API-laget hoster desuden en HTML-baseret webserver, der fungerer som monitoreringsinterface for slutbrugeren. Funktionaliteten er begrænset til visning af tilstandsinformation, da konfigurationsændringer ikke er tilladt på dette niveau.

Driftsprincipper for Edge-enheden

Edge-enheden er konfigureret efter et "minimal workload"-princip, hvor beregningstunge operationer placeres i backend-laget, der har bedre ressourcer til rådighed.[6] Den eneste undtagelse er konvertering af rå Modbus-registre til anvendelige procesværdier.[7]

Denne placering er valgt, fordi normalisering kræver kendskab til enhedsspecifikke registerlayouts og skalafaktorer, og fordi kvaliteten af downstream-data logisk bliver ensartet, når alle systemer modtager ensformigt skalerede værdier, uafhængigt af den individuelle producentens dokumentation.

Use case



Use-casen for systemet omfatter to roller med klart adskilte privilegier.

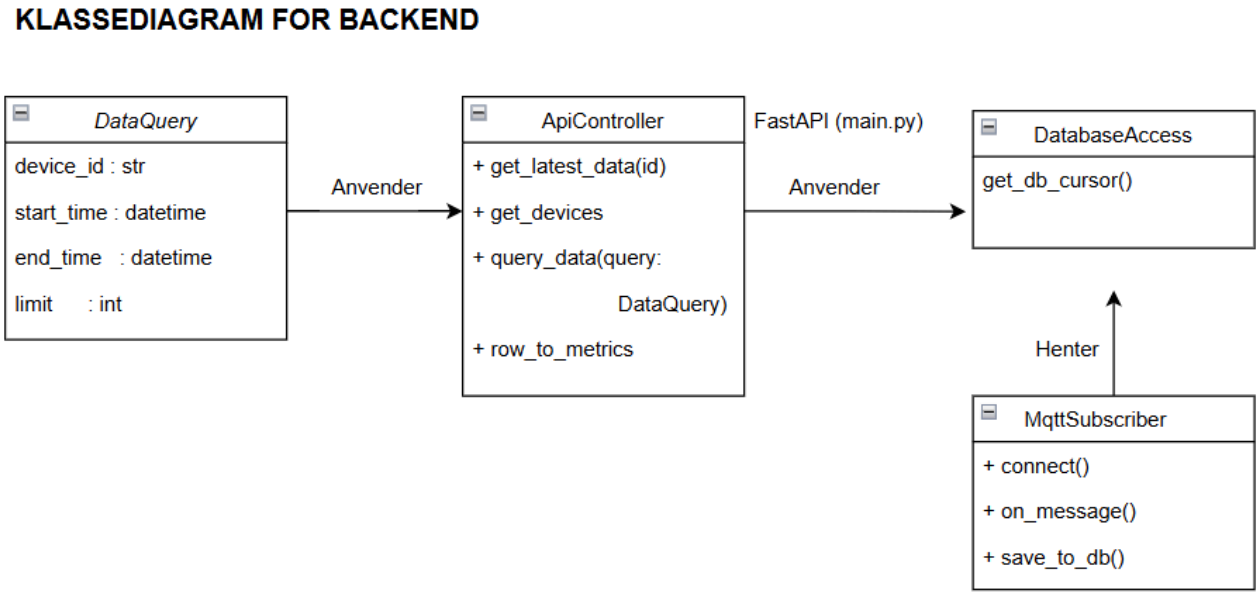
[6] https://arxiv.org/abs/2201.11837
[7] https://azure.microsoft.com/da-dk/resources/cloud-computing-dictionary/what-is-edge-computing

Udvikleren interagerer med backend-laget via Grafana og har udvidede rettigheder til konfiguration, fejlanalyse og vedligehold. Adgangen er tiltænkt udviklingsfunktioner og omfatter privilegier til ændring af Edge-enhedens, MQTT protokollens, databasens, API-lagets og dashboardets konfiguration.

Brugeren interagerer alene med API-lagets webserver og har udelukkende læseadgang. Der gives ingen mulighed for at ændre systemets konfiguration eller ventilationsanlæggets drift. Denne rolleadskillelse følger principperne om minimal privilegietildeling og sikrer, at systemets funktioner og konfiguration forbliver intakte.

Adskillelsen mellem rollerne afspejler den underliggende arkitektur: et edge-lag med datagenerering, et backend-lag med lagring og visualisering, og et API-lag som kontrolleret præsentationsflade for slutbrugeren. Dette understøtter en sikker og reproducerbar interaktion med MVP'ens funktionelle komponenter.

Klassediagram



Klassediagrammet viser opsætningen for backend. I DataQuery agere datamodel for tidligere forespørgsler, den definerer parametre som device_id, tidsintervaler og limit. ApiController anvender DataQuery og omsætter værdierne til forespørgelser med query_data.

DataAccess viser de fælles lag for databaseadgang og åbner og lukker forbindelser til QuestDB. Disse lag bruges af ApiController og MQTT Subscriber til at læse og gemme data. MQTT Subscriber samler funktionerne som behandler MQTT forbindelser og sender beskeder videre til databasen.

Klasser/funktioner i main.py

Der defineret en klasse - class DataQuery(BaseModel)

```
class DataQuery(BaseModel):
    device_id: str
    start_time: datetime
    end_time: datetime
    limit: int = 500  # number of rows (each row becomes many metric points)
```

DataQuery fungerer som en Pydantic-model, der håndhæver strukturen på JSON-payloaden i POST-anmodninger til /api/data/query. [8][9]

Modellen definerer de felter, API'et kræver: device_id som streng, start_time og end_time som tidsstemplede værdier samt et heltal limit, der angiver det maksimale antal rækker, der må returneres. Her anvendes en begrænsning på 500

Der er defineret seks funktioner i main.py:

1.
```
@app.get("/")
def root():
```

- Modtager en HTTP Get request på root URL `"/")`, root håndtere den.

2.
```
def row_to_metrics(ts, row_values):
```

- Database række ændres til metrics i JSON format.

3.
```
@app.get("/api/data/latest/{device_id}")
def get_latest_data(device_id: str):
```

- Finder den nyeste række i databasen, bliver returneret som metrics. Der bliver håndteret GET requests for et bestemt device id.

[8] https://fastapi.tiangolo.com/tutorial/body/
[9] https://docs.pydantic.dev/latest/concepts/models/

4.

```
@app.get("/api/devices")
def get_devices():
```

- Håndterer GET requests for api og devices, slår de forskellige device ide op i databasen og returnerer dem som en liste.

5.

```
@app.post("/api/data/query")
def query_data(data: DataQuery):
```

- Håndter POST request med JSON. DataQuery læser device og tidsinterval og returnerer med historisk data fra databasen.

6.

```
@contextmanager
def get_db_cursor():
```

- Hjælpefunktion som når den kaldes opretter forbindelse til databasen, laver en courser og yielder til den kodeblok der kører inde i with get_db_cursor() as cur:. Når blokken er færdig, sørger funktionen automatisk for at committe ved succes, lave rollback ved fejl og altid lukke både cursor og databaseforbindelse.

Protokoller der er blevet brugt

HTTP Rest.
Least privilege.
MQTT over TCP/IP
FastAPI
PostgreSQL

Industrielle protokoller (IEC, ISO, Sparkplug, osv.)

Sparkplug B

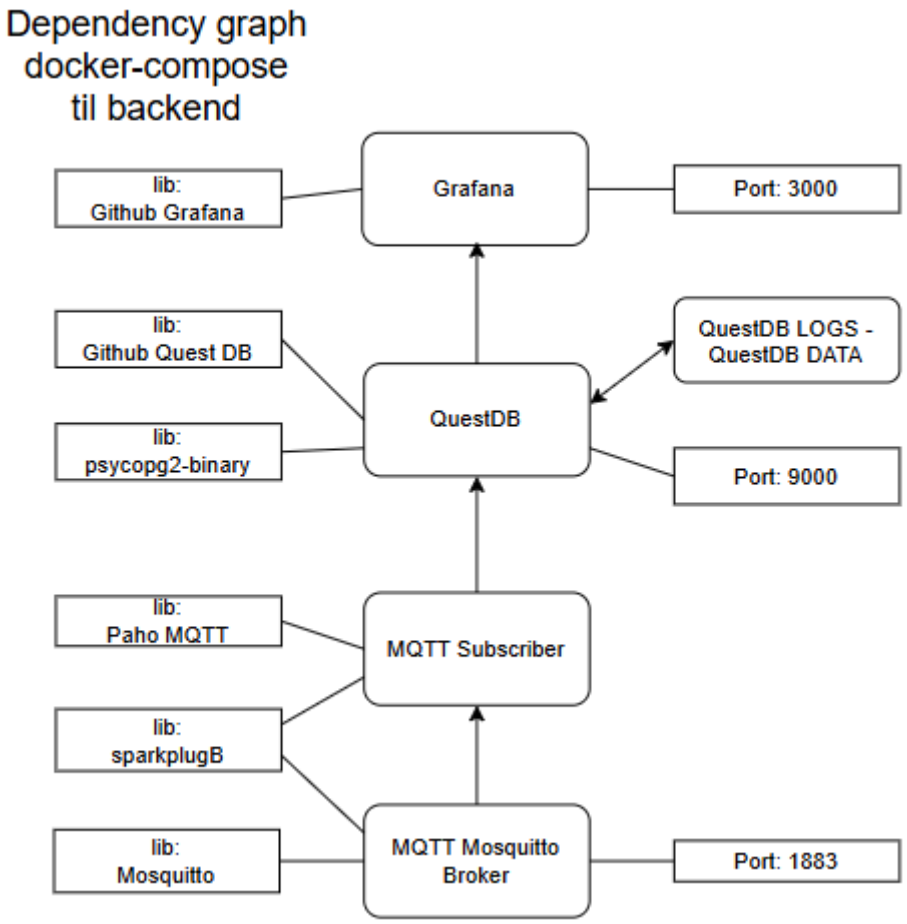
Modbus RTU over RS485

Industrielle Standarder

ISO27002 - Information Security

ISO9001 - Quality Management

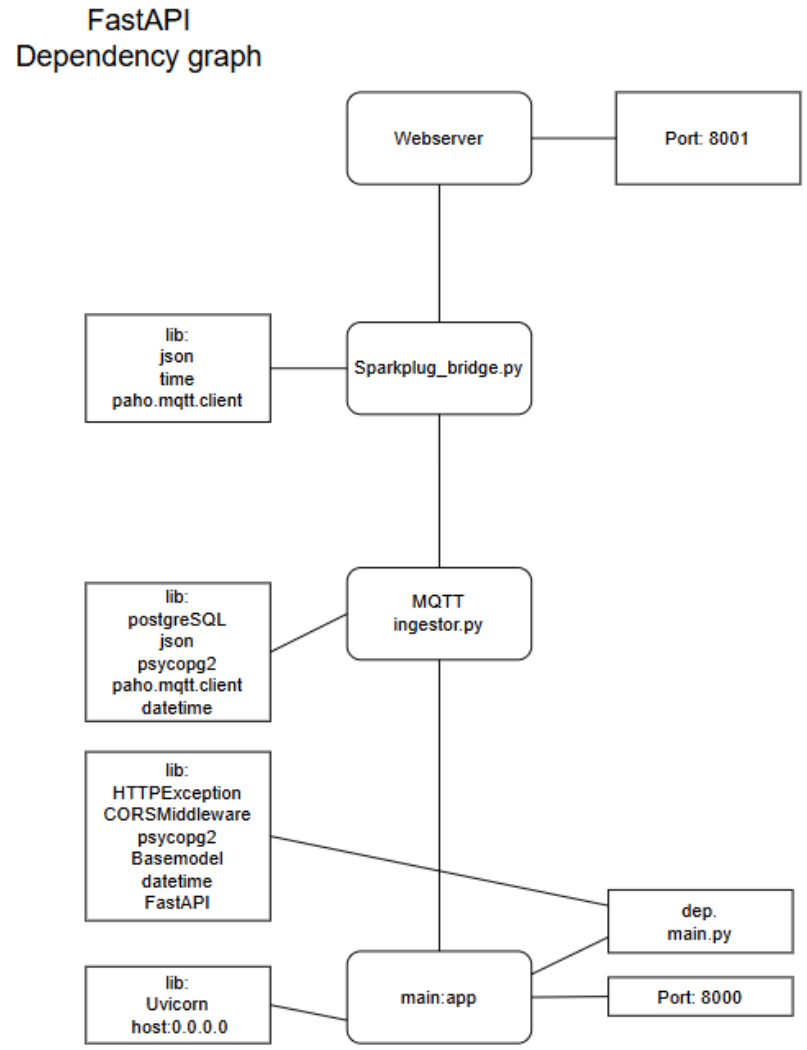
Oversigt docker-compose rækkefølge.



På dependency-grafen ovenfor kan man se de forskellige programmer der kører i backend. Dependency-grafen giver et overblik over de tilhørende porte og libraries som programmerne bruger for at fungere. Grafen giver et solidt indblik i hvordan man kan implementere en stabil deployment sekvens, gennem Docker Compose, særligt når sekvensen betinges af godkendte healthchecks.[10] Healthcheck betinget deployment vurderes til at være out of scope for første iteration af MVP'en, men er et oplagt udviklingspunkt for fremtidige iterationer.

[10] https://docs.docker.com/guides/go-prometheus-monitoring/compose/

Oversigt over manuel FastAPI opstart rækkefølge.



NB: Følgende tekst er forfattet med AI, se bilag 6, Prompt D.

Oversigt over hvordan FastAPI-delen af systemet hænger sammen – fra indsamling af data til HTTP-API'et.

Øverst ligger webserveren, på port 8001. Den bruger filen Sparkplug_bridge.py til at kommunikere med MQTT-broker. Sparkplug_bridge.py bruger bibliotekerne JSON, time og paho.mqtt.client til at modtage Sparkplug-beskeder, oversætte dem til almindelig JSON og sende dem videre.

Næste led er MQTT_ingestor.py. Den abonnerer på de relevante MQTT-emner, pakker måledata ud og gemmer dem i databasen. Derfor afhænger den af PostgreSQL/psycopg2, json, paho.mqtt.client og datetime. På den måde er den bindeleddet mellem MQTT-laget og databasen.

Nederst er selve FastAPI-applikationen main:app, som kører på port 8000 med Uvicorn

(host=0.0.0.0). main.py bruger FastAPI, HTTPException, CORSMiddleware, BaseModel (Pydantic), datetime og igen psycopg2 for at kunne lave HTTP-endpoints, validere data og læse fra den samme PostgreSQL-database, som MQTT_ingestor.py skriver til.

Metode afgrænsning:

Projektets metodiske fokus blev afgrænset til at etablere en fuldt funktionel end-to-end-kæde med live dataflow fra sensor til dashboard.

Denne prioritering medførte, at sekundære aspekter såsom sikkerhedshærdning, stabilitetsoptimering, udvidet logging, systematisk testdækning og avanceret datapræsentation blev nedprioriteret. Afgrænsningen muliggjorde en målrettet udviklingsproces, hvor hovedmålet var at validere den data pipeline og demonstrere en sammenhængende MVP-arkitektur.

5. Resultater og analyse

Præsentation af prototype

Systemoversigt:

MVP løsningen udgør et end-to-end monitoreringssystem bestående af:

Edge plant: Systemair Danvent DV10 med Corrigo E kontroller.
Edge-enhed til dataopsamling: Olimex ESP32-PoE med Modbus RTU master.
Data-transport: Eclipse Mosquitto Broker.
Database: QuestDB tidsseriedatabase.
Backend: FastAPI baseret read only API.
Frontend: Grafana dashboard.

I drift er systemet en pipeline som trækker data fra ventilationsanlæggets sensorer som transporteres, lagres og præsenteres. Se blokdiagram 1 under ‘Teoretisk fundament, Systemet overordnet’ for visualisering af pipelinen.

Edge-laget.

Edge-laget udgør systemets primære interface mod DV10-anlæggets controller (Corrigo E). Implementeringen er baseret på Modbus RTU via en RS-485 linje gennem et A/B leder differentielt signalpar.

Parametrene til Modbus RTU:
Baud Rate = 9600
Modbus Slave ID = 1
RX Pin = 36
TX Pin = 4
DE og RE-signaler kontrolleres manuelt via henholdsvis Pin 5 og Pin 14.

Dette implementeres i edge-enhedens main fil således:

```
#define RX_PIN 36
#define TX_PIN 4
#define MAX485_DE 5
#define MAX485_RE_NEG 14
#define BAUD_RATE 9600
#define MODBUS_SLAVE_ID 1
```

Når forbindelsen er etableret. Læser koden både skalerede registre (temperaturer, tryk, flow) og rå registre (drifttilstand, runtimes).

```
void readAllSensors() {
  Serial.println("\n=== READING SENSORS ===");
  currentData.timestamp = millis();
  currentData.dataValid = false;
  int success = 0;

  if (readScaledReg(1, currentData.heatExchangerEfficiency)) success++;
  if (readRawReg(2, currentData.runMode)) success++;

  readScaledReg(0, currentData.outdoorTemp); if
(!isnan(currentData.outdoorTemp)) success++;
  readScaledReg(6, currentData.supplyAirTemp); if
(!isnan(currentData.supplyAirTemp)) success++;
  readScaledReg(7, currentData.supplyAirSetpointTemp); if
(!isnan(currentData.supplyAirSetpointTemp)) success++;
  readScaledReg(8, currentData.exhaustAirTemp); if
(!isnan(currentData.exhaustAirTemp)) success++;
  readScaledReg(19, currentData.extractAirTemp); if
(!isnan(currentData.extractAirTemp)) success++;

  readScaledReg(12, currentData.supplyAirPressure); if
(!isnan(currentData.supplyAirPressure)) success++;
  readScaledReg(13, currentData.extractAirPressure); if
(!isnan(currentData.extractAirPressure)) success++;

  readScaledReg(14, currentData.supplyAirFlow); if
(!isnan(currentData.supplyAirFlow)) success++;
  readScaledReg(15, currentData.extractAirFlow); if
(!isnan(currentData.extractAirFlow)) success++;
  readScaledReg(292, currentData.extraSupplyAirFlow); if
(!isnan(currentData.extraSupplyAirFlow)) success++;
  readScaledReg(293, currentData.extraExtractAirFlow); if
(!isnan(currentData.extraExtractAirFlow)) success++;

  readRawReg(3, currentData.supplyFanRuntime); success++;
  readRawReg(4, currentData.extractFanRuntime); success++;

  currentData.successfulReads = success;
```

```
  currentData.dataValid = (success > 8);
  Serial.printf("✓ %d/%d sensors OK\n", success, 15);
}
```

For fuld definition af variabler mm. se den fulde Edge-enhed kode, main.cpp, under edge/src mappen.

MQTT

MQTT-kommunikationen er baseret på **TLS 1.2**, hvilket etableres gennem WiFiClientSecure. Root CA-certifikatet er indlejret i programkoden og anvendes som trust anchor:

```
WiFiClientSecure espClient;

espClient.setCACert(root_ca);

PubSubClient client(espClient);
```

Denne opsætning medfører, at brokerens certifikat verificeres ved forbindelsesoprettelse. Derved undgås ukrypterede forbindelser og man-in-the-middle-angreb, hvilket er en standardmetode i IoT-systemer med offentlig netværksadgang.[11]

Kommunikationen udstilles herefter til MQTT-klienten:

```
client.setServer(mqtt_server, 8883);
```

Port 8883 er defineret som standardport for MQTT over TLS og muliggør kompatibilitet med eksisterende broker-implementeringer såsom Mosquitto.

- Sparkplug B: NBIRTH og DBIRTH

Sparkplug B anvendes til at beskrive livscyklussen af datablokken. Dette giver forudsigelig opførsel, når backend-laget skal fortolke data fra edge-laget.[12]

NBIRTH publiceres ved første etablerede forbindelse:

```
void publishBirthMessages() {

    client.publish("spBv1.0/VENT/NDATA/NBIRTH",

                   "{ \"online\": true }");

}
```

DBIRTH beskriver hvilke datapunkter enheden vil publicere fremadrettet:

```
client.publish("spBv1.0/VENT/DDATA/DBIRTH",
```

[11] https://medium.com/@kajsuaning/mqtt-mutual-certificate-authentication-f51bc6e1a457
[12] https://i-flow.io/en/ressources/what-is-sparkplug-b-pros-and-cons-of-the-standard/

```
                "{ \"metrics\": [\"temp_out\", \"temp_in\",
\"pressure_pa\", \"fan_mode\"] }");
```

Backend-systemer kan herved validere, at de modtagne datapunkter er komplette og matcher den forventede struktur.

Efter hver Modbus-polling behandles de rå registerværdier. Skalering gøres lokalt på Edge-enheden, da registerlayoutet kun er entydigt tilgængeligt ved Modbus-endpointet. De normaliserede værdier konverteres til JSON:

```
StaticJsonDocument<256> doc;

doc["temperature_out"] = tempOut;

doc["temperature_in"]  = tempIn;

doc["pressure_pa"]     = pressurePa;

doc["fan_mode"]        = fanMode;

char buffer[256];

serializeJson(doc, buffer);
```

Ingestor og Data-transport

MQTT-ingestoren udgør bindeled mellem edge-laget og database-laget. Implementeringen er baseret på Paho MQTT-klienten og fungerer som subscriber til broker-topics, hvor den modtager sensordata og persisterer disse i QuestDB. Ingestoren etablerer sikker kommunikation med Mosquitto-brokeren via TLS 1.2 på port 8883. Autentifikation sker gennem username/password credentials, og CA-certifikatet verificeres ved forbindelsesoprettelse:

```
client = mqtt.Client(client_id="QuestDB_Ingestor", protocol=mqtt.MQTTv5)

client.username_pw_set(MQTT_USER, MQTT_PASS)

client.tls_set(ca_certs=CA_CERT_PATH)

client.connect(MQTT_HOST, MQTT_PORT, 60)
```

API-laget

API-laget udgør systemets API-interface og leverer struktureret adgang til de indsamlede tidsseriedata. Implementeringen er baseret på FastAPI.

Backenden eksponerer tre primære endpoints:

- /api/devices returnerer liste over registrerede enheder

- /api/data/latest/{device_id} henter seneste måling

- /api/data/query understøtter historiske forespørgsler via POST

Forbindelsen til QuestDB etableres gennem psycopg2-driveren, som kommunikerer via QuestDB's PostgreSQL-kompatible wire protocol på port 8812.

```
def create_db_connection(max_retries=10):

    """Etablerer forbindelse til QuestDB ved hjælp af psycopg2."""

    for attempt in range(max_retries):

        try:

            db_conn = psycopg2.connect(

                database="qdb",

                user="admin",

                password="quest",

                host=QDB_HOST,

                port=QDB_PORT

            )

            logger.info("QuestDB dataforbindelse etableret
succesfuldt.")

            return db_conn

        except psycopg2.OperationalError as e:

            logger.warning(f"QuestDB dataforbindelse forsøg
{attempt+1}/{max_retries} mislykkedes: {e}")

            if attempt < max_retries - 1:

                time.sleep(5)

            else:

                logger.critical("QuestDB dataforbindelse mislykkedes
efter maksimum forsøg.", exc_info=True)

                raise
```

Forbindelsesparametrene er defineret i DB_CONFIG:

```
DB_CONFIG = {
    "dbname": "qdb",
    "user": "admin",
    "password": "quest",
    "host": "localhost",
    "port": 8812
}
```

Logging-system

Systemet implementerer struktureret logging med dual output: console logging for udvikling og database logging for produktion. Ved startup oprettes en dedikeret logging-tabel i QuestDB, hvor alle API-requests, fejl og systemhændelser opbevares:

```
@app.on_event("startup")
async def startup_event():
    create_logging_table(DB_CONFIG)
    logger.info("FastAPI server starting up...")
```

For fuld API-kode og endpoint-definitioner, se main.py fil under backend mappen.

Frontend-visualisering

Frontend-laget implementeres gennem Grafana. Grafana kører som containeriseret service og eksponerer web-interface på port 3000:

```
grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    depends_on:
```

```
      - questdb

    networks:

      - iot-net
```

Dataintegration

Grafana forbinder til FastAPI-backenden som datakilde gennem det fælles iot-net Docker-netværk. depends_on-direktivet sikrer, at QuestDB er startet før Grafana, hvilket forhindrer fejl ved første opstart når Grafana forsøger at validere datakilde-forbindelser.

Dashboardet er konfigureret til at query API-endpointerne for både real-time data (/api/data/latest/{device_id}) og historiske tidsserier (/api/data/query).

Testprotokol

Formålet med testene er at verificere, at systemets centrale softwarekomponenter fungerer korrekt uden afhængighed af fysisk hardware eller eksterne tjenester.

NB: Alle tests anvender mocking til at erstatte netværk, sensorer og databaseforbindelser med kontrollerbare testobjekter.[13]

Følgende test filer er blevet udformet til projektet:

1. **test_edge_client_logic.py (Edge-enheden)**

Testene validerer styringslogikken på edge-enheden ved at simulere hardware og netværksforhold. I disse er WiFi-modul og MQTT-klient erstattes af mock-objekter, så forbindelseslogik kan testes løbende.

Testene dækker manglende WiFi-initialisering, ugyldige MQTT-loginoplysninger og afbrudte forbindelser. Dette sikrer at koden på Edge-enheden kan identificere og handle på netværksfejl.

2. test_main.py (API-laget)

Testene verificerer API-endepunkternes funktionalitet uden at starte en reel server.

Forespørgsler mod den faktiske database erstattes af mock-svar, som gør det muligt at teste API-logikken isoleret.
FastAPI's TestClient bruges til at afsende GET- og POST-anmodninger.
Dermed kan det bekræftes, at API'et returnerer korrekte svar under normale forhold og leverer konsistente fejlmeddelelser ved databasefejl.[14]

3. test_mqtt_ingestor.py (Datahåndteringsservice)

Testene kontrollerer robustheden af datahåndteringstjenesten, der modtager og indsætter datapakker i databasen.

Midlertidige databasefejl simuleres for at verificere, at tjenesten udfører gentagne forsøg og gennemfører indsættelsen.

Mockede MQTT-beskeder anvendes til at kontrollere, at den genererede SQL-kommando matcher det forventede format.

Funktionalitet af prototypen

Jævnfør de etablerede kravspecifikationer kan følgende resultater for prototypen fremvises:

Krav 1 - Dataudtræk

Krav: Edge-enheden kan lave dataudtræk fra alle relevante Modbus-registre.

Resultat: Edge-enheden fungerer og kan læse 15 ud af 15 registre. Se bilag 1 for oversigt over relevante Modbus-registre og bilag 2 for eksempel på dataudtræk.

[13] https://docs.python.org/3/library/unittest.mock.html
[14] https://fastapi.tiangolo.com/tutorial/testing/

Krav 2 - Modbus råværdier

Krav: Edge-enheden kan konvertere Modbus råvædier til brugbare værdier for teknisk personale.
Resultat: Edge-enheden konverterer råværdier korrekt, se bilag 2 for serial print af både råværdi og korrekt værdi.

Krav 3 - Overenstemmelse med reference-enhed

Krav: Edge-enheden angiver værdier som stemmer med værdierne vist i DV10 anlæggets reference-enhed, inden for 10% tolerance.
Resultat: Reference-enheden og Edge-enhedens værdier stemmer overens inden for tolerancen, se bilag 3 for dokumenteret måling.

Krav 4: MQTT Kommunikation

Krav: MQTT Broker acceptere forbindelser fra Edge enhed og backend, og udfører dataoverførsel fra MQTT publisher på Edge enheden, til MQTT subscriber på backend, med latency under 500ms.
Resultat: Systemet opretholder stabil MQTT kommunikation med latency under kravet. Se bilag 4.

Krav 5 - QuestDB tabel

Krav: QuestDB indeholder mindst én tabel til ventilationsdata med timestamp.
Resultat: Tabellen oprettes og modtager data med timestamp, se bilag 5.2.

Krav 6 - QuestDB Tidseriedata

Krav: QuestDB lagrer data som tidsseriedata fra ventilationsanlægget.
Resultat: QuestDB fremviser data i en tidsrække som forventet, se bilag 5.8

Krav 7 - FastAPI endpoints

Krav: FastAPI eksponerer endpoints, som henter data fra QuestDB.
Resultat: Alle definerede endpoints fungerer og henter data via sikre databaseforbindelser. Koden bruger get_db_cursor() konsistent. Se API-koden i bilag 6.

Krav 8 - MQTT Logs

Krav: Systemet leverer logs i SPDlogs-format vedrørende MQTT Broker.
Resultat: Krav 8 opfyldes ved, at systemet automatisk opsamler logs fra MQTT-klienten og dataindsættelseslogikken, som gemmes i QuestDBs logging-tabel med kolonnerne ts, logger, level og message for at sikre et SPDlogs-kompatibelt og søgbart overblik. Se bilag 5.10

Krav 9 - API Logs

Krav: Systemet leverer logs i SPDlogs-format vedrørende API-lag webserver.

Resultat: Logningstabellen indeholder aktive INFO beskeder fra backend.main, hvilket beviser, at log-håndteringen er etableret og funktionel. Se Bilag 5.9

Krav 10 - Dashboard visualisering.

Krav: Grafana dashboard visualiserer seks datapunkter fra ventilationsanlægget.

Resultat: Dashboardet visualiserer mindst ni live-datapunkter, herunder temperaturer, tryk, flow og driftsstatus. Se bilag: 5.3.

Krav 11 - Webserver live-data

Krav: Webserver på API-laget kan fremvise live datasæt med runmode, runtime, temperatur, tryk og flow fra DV10 anlægget.

Resultat: Webserver fremviser påkrævede data i realtid som specificeret. Se bilag 5.1.

Krav 12 - MQTT-payload

Krav: MQTT-payload er JSON-formateret og bruger topic-struktur og meddelelsestyper formateret efter Sparkplug B's principper. (NBIRTH, DBIRTH).

Resultat: MQTT-payloads opfylder strukturkravet. Topic- og meddelelsesformat er konsistent med Sparkplug B's principper. Dokumentation i Edge-enhedens kode under mappen edge.

Krav 13 – Unit tests for centrale komponenter

Krav: Unit tests skal være defineret for Edge-enheden samt for systemets backend komponenter: MQTT-broker, QuestDB, FastAPI og Grafana.

Resultat: Unit tests er implementeret for alle nævnte komponenter. Testoversigt i bilag.

Krav 14 – Beståede unit tests

Krav: Alle implementerede unit tests skal bestås.
Resultat: Samtlige tests er bestået uden fejl. Testrapport i bilag.

6. Diskussion

Praktisk anvendelighed

Prototypen demonstrerer, at en fuld IoT-MVP fra sensor til dashboard kan realiseres med en relativt simpel arkitektur. API'et er dog begrænset i funktionalitet og skalerer ikke uden videre; det tilbyder kun læseadgang og ingen mekanismer for brugerkontrol, filtrering eller avanceret datasøgning.

Alligevel giver løsningen et solidt iterationsfundament: Edge-laget er funktionelt, backend-kæden er gennemgående stabil, og systemet leverer live-data med lav latenstid. Fremtidige iterationer kan fokusere på stabilitet, sikkerhed, standardisering og reel brugerafprøvning.

Under udviklingen blev der truffet en beslutning at opdele systemet i to miljøer. En række opstartsfejl i Docker — hvor API'en forsøgte at etablere en databaseforbindelse, før QuestDB var klar — udløste hyppige nedbrud, som gjorde fejlsøgning unødigt langsom.

Se dependency graph for Docker Compose og dependency graph for API-laget som reference.

Den praktiske løsning blev at lade de stabile komponenter (QuestDB, Mosquitto, Grafana) køre i Docker Compose, mens den aktive udviklingskode (API, Ingestor, Sparkplug-Bridge) kører lokalt. Denne løsning fjernede fejlkilder uden at ændre den samlede systemarkitektur.

Som konsekvens er den nuværende arkitektur ikke en-til-en reproducerbar uden for udviklingsmaskinen, da løsningen ikke er containeriseret.

Opsummeret:

Det, som virker

Basal end-to-end funktionalitet.

- Edge-enhedens Modbus Master konfiguration kan læse alle relevante registre.
- MQTT kommunikation er stabil.
- Backend kan modtage, lagre og fremvise data.
- Webserver fremviser data relevant for slutbruger.

Det, som ikke virker

- Ingen automatiseret deployment.
- API-laget har begrænset funktionalitet (ingen write-endpoints, ingen filtre, ingen bruger kontrol)
- MQTT kommunikation er ikke Sparkplug B complaint.
- Data pipelinenen er afhængig af én VM uden redundans, hvilket udgør en driftsrisiko.
- Ingen persistent logs ved nedbrud.
- Webserver funktionalitet er begrænset da den er placeret i backend, men skal betjenes af slutbruger i edge-laget.
- Systemet er ikke konfigureret i henhold til ISO standarder eller best practises

Selvom prototypen fungerer, er den ikke driftsstabil. Den kan bruges til test og udvikling, men ikke til kontinuerlig overvågning uden risiko for nedbrud.

Set fra et udviklingsperspektiv vurderes MVP'en som tilstrækkelig til at demonstrere systemets gennemførlighed og pipeline, men ikke som en driftsstabil løsning.

Udfordringer og muligheder

Udfording: Single point of failure.

Debian 12 maskinen udgør et punkt hvor næsten samtlige funktioner for TCP/IP netværket og datapipelinen er samlet. Det gør opsætningen let at udvide, men gør samtidig systemet sårbart og skrøbeligt, da hele systemet afhænger af at serveren er fuld funktionel.[15]

Muligheder inden for kort tidshorisont.

På kort sigt er der lavthængende forbedringer. Flere edge-cases bør testes for at afdække, hvordan systemet håndterer uventede input og netværksforstyrrelser.

MQTT-publisheren på Edge-enheden bør også opdateres til en nyere JSONArduino-standard; den nuværende version (6) genererer et stort antal compiler-advarsler og er ikke hensigtsmæssig for vedligeholdbar kode.

Derudover bør logging-strategien udvides: både Olimex ESP32-PoE Edge-enheden og backend-servicen kunne logge lokalt (fx via SD-kort) for at give bedre indblik i fejl og edge cases.

7. Perspektivering

Identifikation af udviklingsmuligheder

Standardisering og skalering

Selvom MVP'en viser, at end-to-end-arkitekturen fungerer, er den langt fra klar til skalering. Sparkplug B og protobuf var planlagt, men ikke implementeret, og uden en formel datamodel bliver systemet svært at udvide til flere enheder eller større installationer.

En fuld standardisering af datapipelinen er derfor et naturligt næste skridt, især hvis løsningen skal kunne indgå i industriel produktion.[16]

Infrastruktur og driftsikkerhed

[15] https://en.wikipedia.org/wiki/Single_point_of_failure
[16] https://www.eclipse.org/tahu/spec/sparkplug_spec.pdf

Hele systemet afhænger af én Debian-VM, og hvilket er en løsning udelukket egnet til udvikling. En fremtidig version bør flyttes til cloud-infrastruktur, hvor automatiseret deployment og redundans er en del af det grundlæggende.[17] Mange af MVP'ens praktiske begrænsninger, herunder manglende stabilitet, sårbar sikkerhed og afhængigheden af lokale services, forsvinder sandsynligvis først, når driften migreres ud af udviklingsmaskinen.

Edge-enheden og OTA

Firmwareudviklingen på ESP32-PoE blev gennemført manuelt, og OTA-opdateringer nåede aldrig længere end planlægningen.

OTA med rollback-mekanismer er afgørende, før edge-enheden kan betragtes som driftsklar, især hvis systemet senere skal skaleres til flere installationer eller forskellige miljøer.[18]

Datakvalitet og robusthed

Flere målinger returnerede fejl, og ingestion-laget behandler dem ikke systematisk. En mere stabil løsning kræver validering, buffering ved netværksudfald og persistent logging både på edge[19] og backend.

MVP'en fungerede kun så længe der ikke opstod problemer; næste iteration bør fokusere på at håndtere fejl.

Udvidelse af webserver funktionalitet

Hvis systemet skal bredes ud til andre enheder eller tilbydes til en kunde, skal API'et udvikles væsentligt. Historiske forespørgsler, write-endpoints, alarmering og brugerroller var alle funktioner, der lå uden for scope for MVP'en, men som bliver essentielle i praktisk drift.

Et webbaseret konfigurations-lag på ESP32-enheden, hvor brugeren selv kan vælge Modbus-registre og poll-frekvenser, ville ligeledes gøre platformen langt mere fleksibel og brugbar for kundens teknikere med ansvar ventilationsanlægget.

Fremtidige udfordringer og løsninger

Ved en senere udrulning af systemet forventes tre hovedudfordringer:

- Driftsstabilitet,

Systemets afhængighed af en enkelt VM vurderes at udgøre den primære driftsrisiko. Denne risiko kan reduceres gennem containerisering af alle komponenter, etablering af automatiseret deployment samt anvendelse af redundante backend-miljøer, eksempelvis i en cloud-baseret infrastruktur.

- Datakvalitet

[17] https://softteco.com/blog/how-to-build-a-cloud-migration-strategy
[18] https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/ota.html
[19] https://developer.espressif.com/blog/2025/09/espressif_logging/

Datakvaliteten vurderes utilstrækkelig til kontinuerlig overvågning, da flere målepunkter returnerer fejl frem for gyldige værdier. Dette kan afhjælpes ved implementering af valideringslag i ingestion-modulet, fallback-logging på edge-enheden samt en systematisk testprotokol, der adresserer måleusikkerhed og transmissionsfejl.

- Standardisering.

Skalerbarhed mod flere enheder forudsætter øget standardisering. Manglende Sparkplug-B-compliance, uensartet topic-struktur og begrænset API-funktionalitet vil hindre udvidelse til større systemer. Disse forhold kan løses gennem adoption af binære payloads (Protocol Buffers), etablering af konsekvente navnepolitikker og en API-struktur, der understøtter filtrering, historiske forespørgsler og write-operationer.

Samlet peger disse forhold på behovet for en mere reflekteret arkitektur, hvor dokumenterede konfigurationer, en udbygget testprotokol og klare standarder danner grundlaget for et driftsklart IoT-system.

8. Referencer

Se fodnoter i dokumentet.

Bilag

Bilag 1 – Udvidet komponentliste

NB: Udvidet komponentliste er AI genereret. Se Bilag 6, Prompt B.

Hardware

Edge

- Olimex ESP32-PoE microcontroller.
- MOD-RS485 interface modul (til RS485/Modbus-kommunikation).
- Micro-USB strømforsyning til ESP32-POE (5V).
- Systemair Danvent DV10 ventilationsanlæg.
- Systemair Corrigo E controller (Modbus RTU slave).

Backend

- Lenovo ThinkPad T490 (host-maskine).
- VMware Workstation 17 Pro (virtualiseringsplatform).
- VM Debian 12 (Backend-server).

Netværk

- iPhone 14 Hotspot (WiFi access point, DHCP). Subnet: 172.20.10.0/28 (tildelt automatisk af hotspot).
- WiFi-adapter i ThinkPad (forbindelse til AP).
- VMNet0 bridged interface (for netværksbro mellem VM og fysisk net).

Software

System- og netværkskomponenter

- Eclipse Mosquitto MQTT broker.
- FastAPI framework (API-hosting + webserver).
- QuestDB tidsseriedatabase (PostgreSQL ingestion og Web UI).
- Grafana dashboard.

- Debian 12 (backend server-OS).
- Windows 11 (host OS)..

Udviklingsværktøjer

- Visual Studio Code.
- PlatformIO.
- Pioarduino (integrationsværktøj til ESP32-projekter).
- Doxygen (kode– og API-dokumentation).

Biblioteker og udviklingskomponenter

- Arduino-ESP32 core (ESP32 biblioteksamling).
- ModbusMaster.h (Modbus klientbibliotek til C++).
- SPDlog (logging i backend).
- Paho MQTT (Python MQTT-klient).
- FastAPI-deps: Requests, uvicorn, psycop2 m.fl.
- Docker Engine.
- Docker Compose (docker-compose.yml).

Programmeringssprog og projektfiler

C++ (Edge-enhed)

- main.cpp → Modbus RTU Master, MQTT publisher.
- Sparkplug B JSON strukturer til dataprotokoller.

Python3 (Backend)

- main.py → FastAPI server.
- mqtt_ingestor.py → MQTT-forbruger og datainsertion til QuestDB.
- simulate_sensors.py → Sensor-simulator til test.

Logging via SPDlog og Python's logging subsystem.

Shell / scripting

- quick_check.sh → Systemstatus for services i Debian VM.

YAML

- docker-compose.yml → Containerorchestration for Mosquitto, Grafana og QuestDB.

HTML / Frontend

- index.html → Indhold til ESP32-webserverens frontend.

Protokoller og standarder

- Modbus RTU (RS485 differential bus).
- MQTT v3.1.1 (dataoverførsel).
- HTTP/REST (API-endpoints).
- TCP/IP (transportlag).
- JSON (datapayloads mellem edge og backend).
- DHCP (adressetildeling fra iPhone hotspot).

Bilag 2:

Adresseregister over anvendte Modbus RTU registernumre.

NB: Angives som adresser anvendt til Modbus Master koden.

Input reg. nr.	Navn	Værdi	Betegnelse på referenceenhed
0	Outdoor Temp	°C	Ude Temp
1	Heat Exchanger Efficiency	%	Virkningsgrad genvinding
2	Run Mode	0-11	Manuel red. hast.
3	Supply Air Fan Runtime	Timer	Drifttid tilluft
4	Extract Air Fan Runtime	Timer	Drifttid fraluft
6	Supply Air Temp	°C	Tillufttemp.
7	Supply Air Setpoint Temp	°C	Tillufttemp. Setp.
8	Exhaust Air Temp	°C	Fralufttemperatur
12	Supply Air Pressure	Pa	Trykregistrering tilluft
13	Extract Air Pressure	Pa	Trykregistrering fraluft
14	Supply Air Flow	m3/h	
15	Extract Air Flow	m3/h	
19	Extract Air Temp	°C	Afkasttemp.
292	Extra Supply Air Flow	m3/h	Ekstra flow SAF
293	Extra Extract Air Flow	m3/h	Ekstra flow EAF

Bilag 3:

Udtrukket data fra serial monitor:

```
╔═══════════════════════════════════════════════════════════════╗
║          READING ALL SENSORS                                  ║
```

```
--- System Status ---
  Heat Exchanger Efficiency [Reg   1]:   343 (34.3 %)
  Run Mode                  [Reg   2]:    5 (Normal run)

--- Temperatures ---
  Outdoor Temp              [Reg   0]:    49 (4.9 °C)
  Supply Air Temp           [Reg   6]:    85 (8.5 °C)
  Supply Air Setpoint Temp  [Reg   7]:    30 (3.0 °C)
  Exhaust Air Temp          [Reg   8]:   205 (20.5 °C)
  Extract Air Temp          [Reg  19]:   156 (15.6 °C)


--- Pressures ---
  Supply Air Pressure       [Reg  12]:   424 (42.4 Pa)
  Extract Air Pressure      [Reg  13]:   583 (58.3 Pa)

--- Air Flows ---
  Supply Air Flow           [Reg  14]:   619 (61.9 m³/h)
  Extract Air Flow          [Reg  15]:   726 (72.6 m³/h)
  Extra Supply Air Flow     [Reg 292]:  2521 (252.1 m³/h)
  Extra Extract Air Flow    [Reg 293]:  2156 (215.6 m³/h)

--- Runtime ---
  Supply Air Fan Runtime    [Reg   3]:   722 (72.2 hours)
  Extract Air Fan Runtime   [Reg   4]:   721 (72.1 hours)

╔═══════════════════════════════════════════════════════════════════════════╗
║  Total: 15/15 successful reads in 1171ms                                  ║
╚═══════════════════════════════════════════════════════════════════════════╝
```

Bilag 4

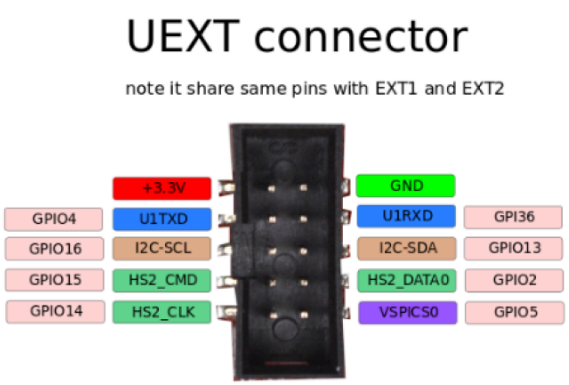
Olimex ESP32-Poe og MOD-RS485 Pins

4.1 UEXT

2.6. UEXT connector:

UEXT connector stands for Universal EXTension connector and typically contains +3.3V, GND, UART, I2C, SPI signals.

UEXT is 0.1” 2.54mm step boxed plastic connector. All signals are with 3.3V levels.



Olimex has developed number of extension MODULES compatible with this connector. There are temperature, humidity, pressure, magnetic field, light sensors. Modules with LCDs, LED matrix, relays, Bluetooth, Zigbee, Wi-Fi, GSM, GPS, RFID, RTC, EKG, sensors, and more.

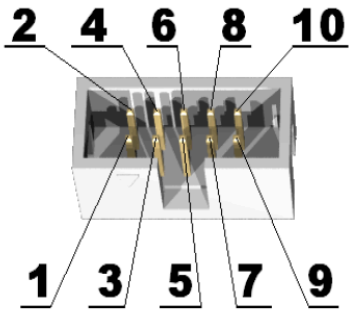
4.2

RS485 pinout billeder

EXTERNAL CONNECTORS DESCRIPTION:

UEXT:

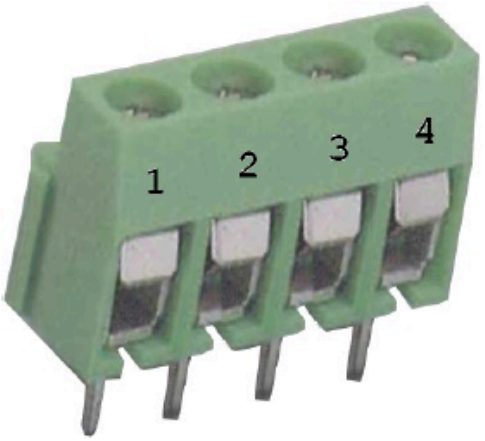
Pin #	Signal Name
1	VCC
2	GND
3	TXD
4	RXD
5	SCL
6	SDA
7	Not connected
8	Not connected
9	SCK
10	#SS



4.3

RS485:

Pin #	Signal Name
1	VCC
2	100 Ohm pull-down resistor
3	A
4	B

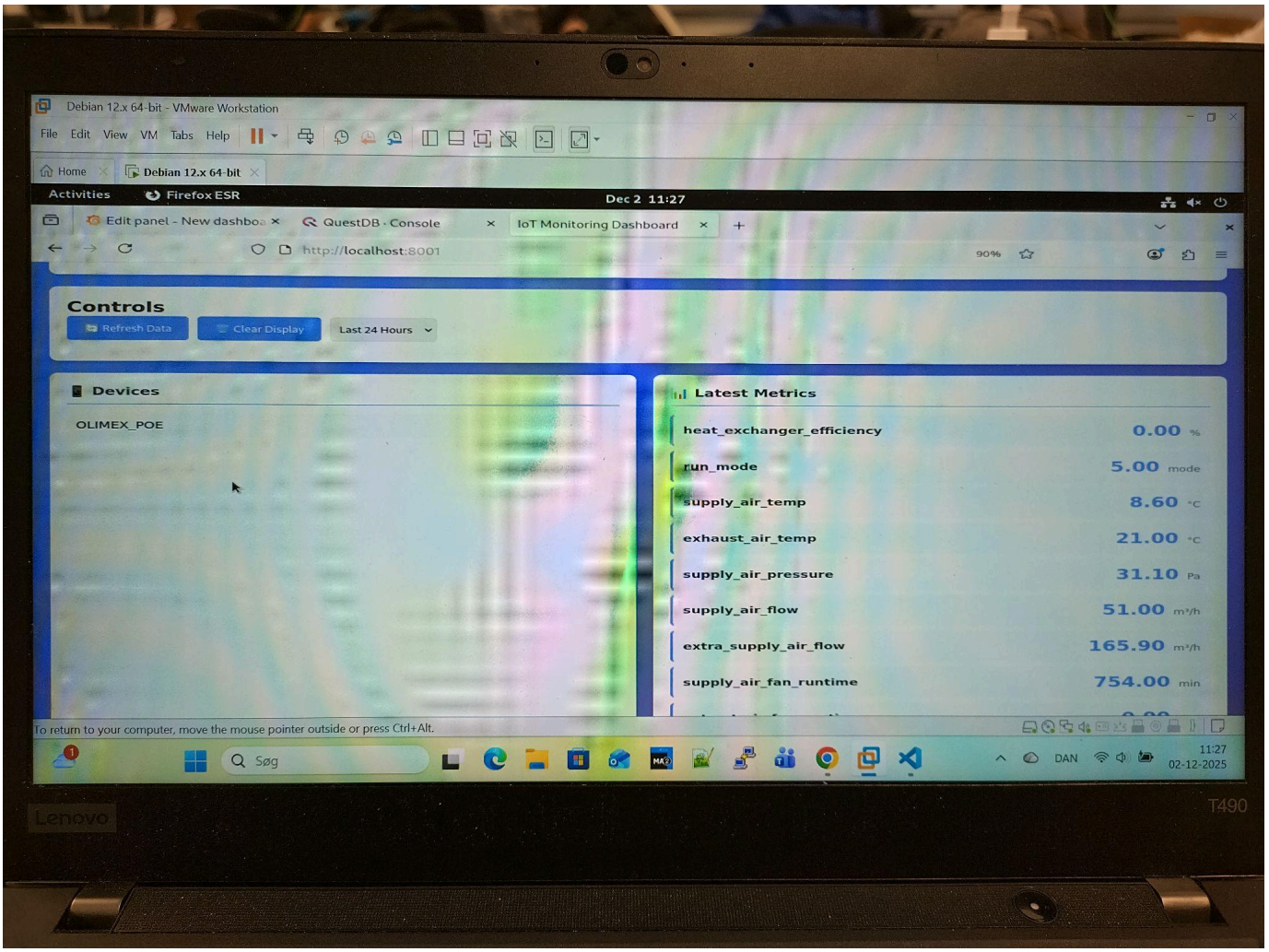


Bilag 5

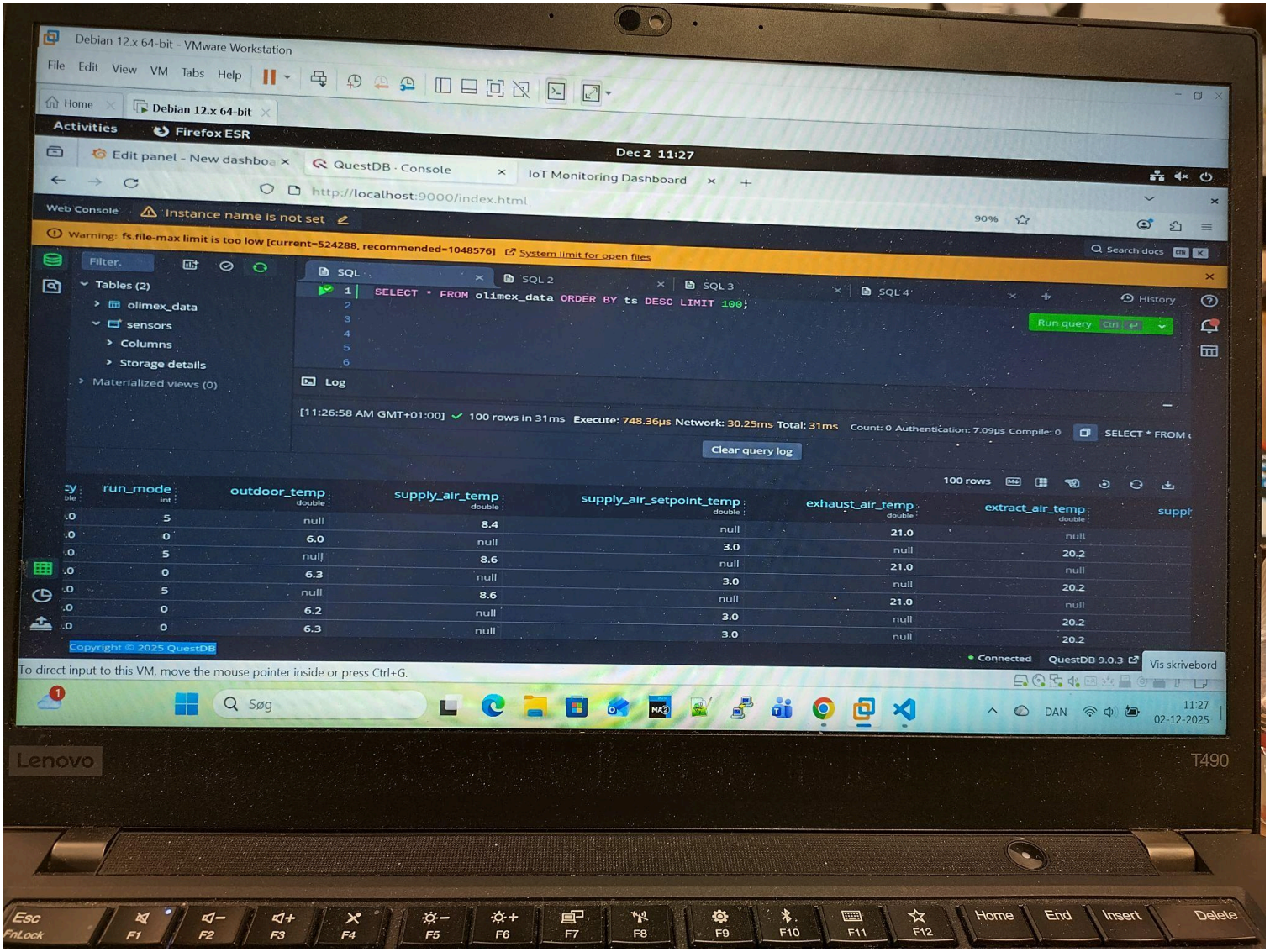
Screenshot dokumentation

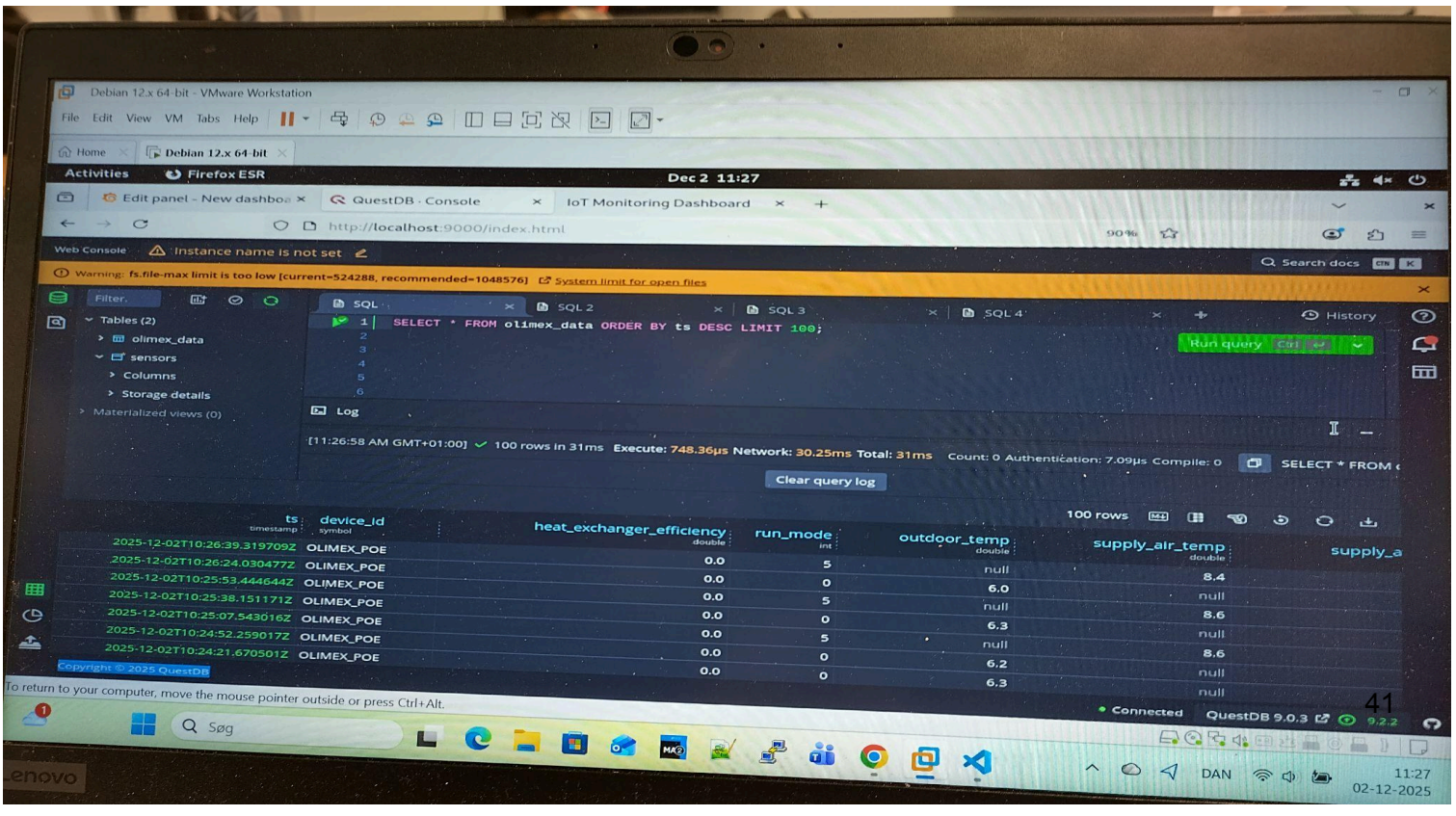
5.1

FastAPI Webserver på port 8001 med live data.

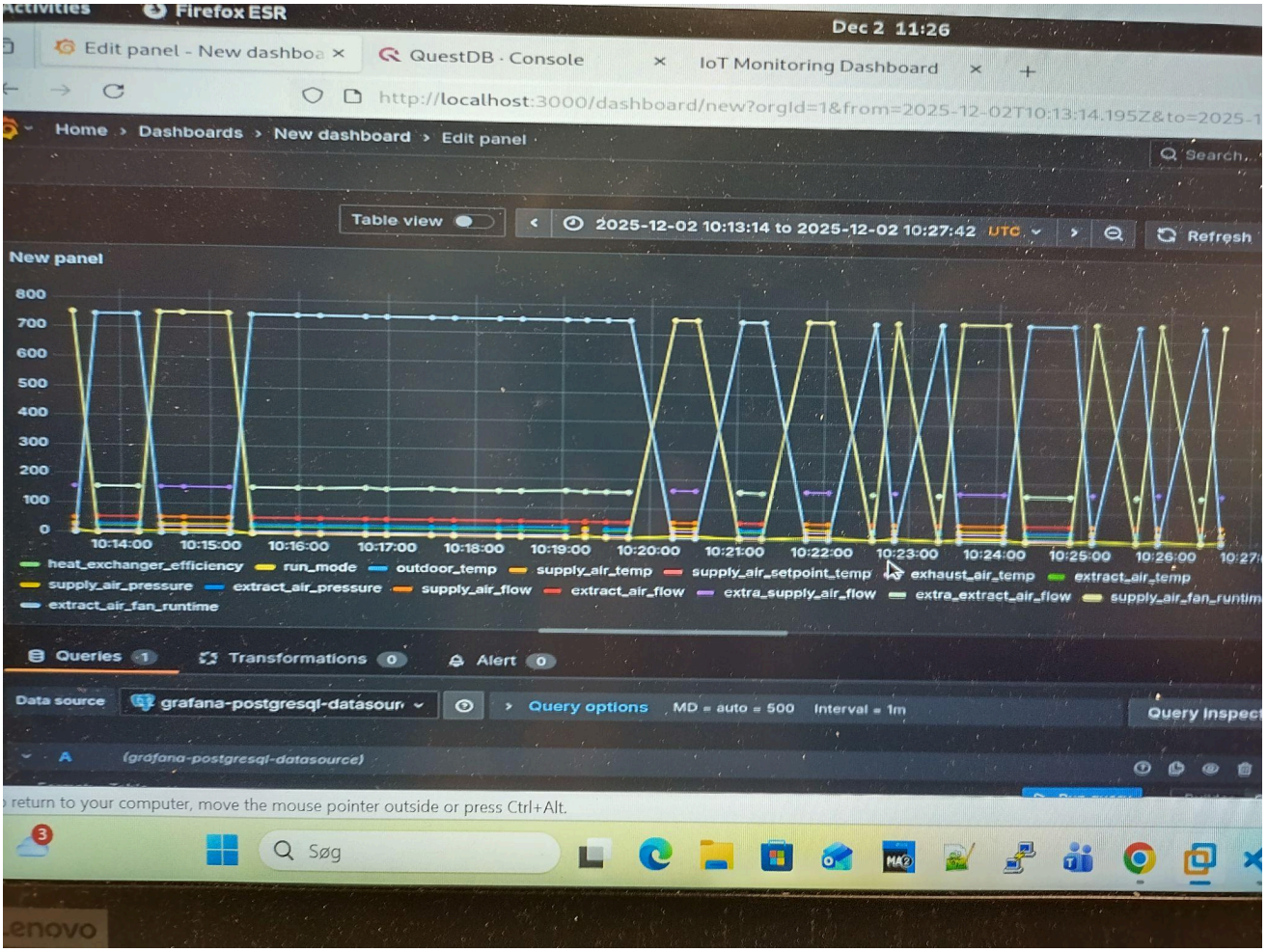


5.2 QuestDB med live data





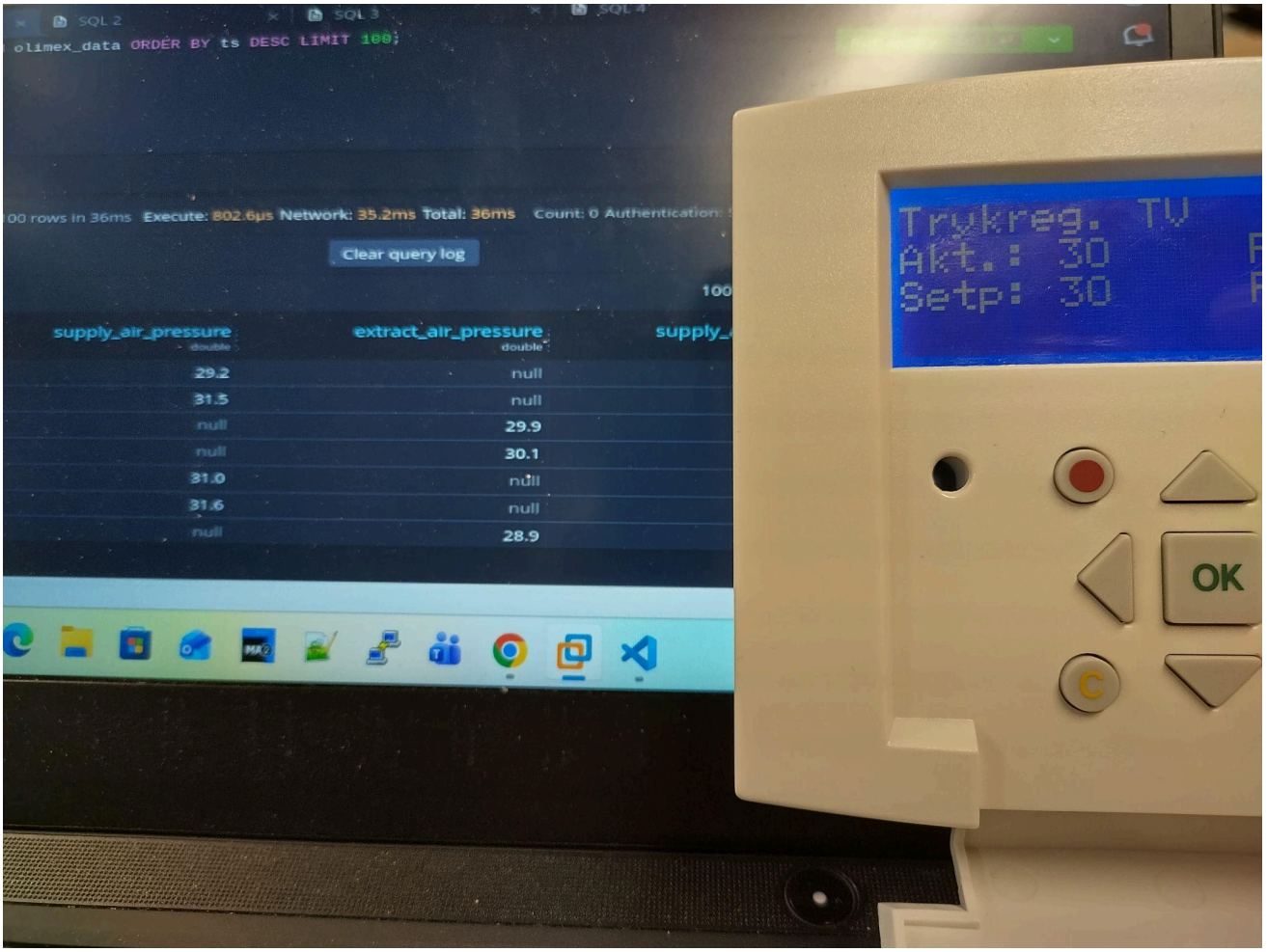
5.3 Grafana med live data



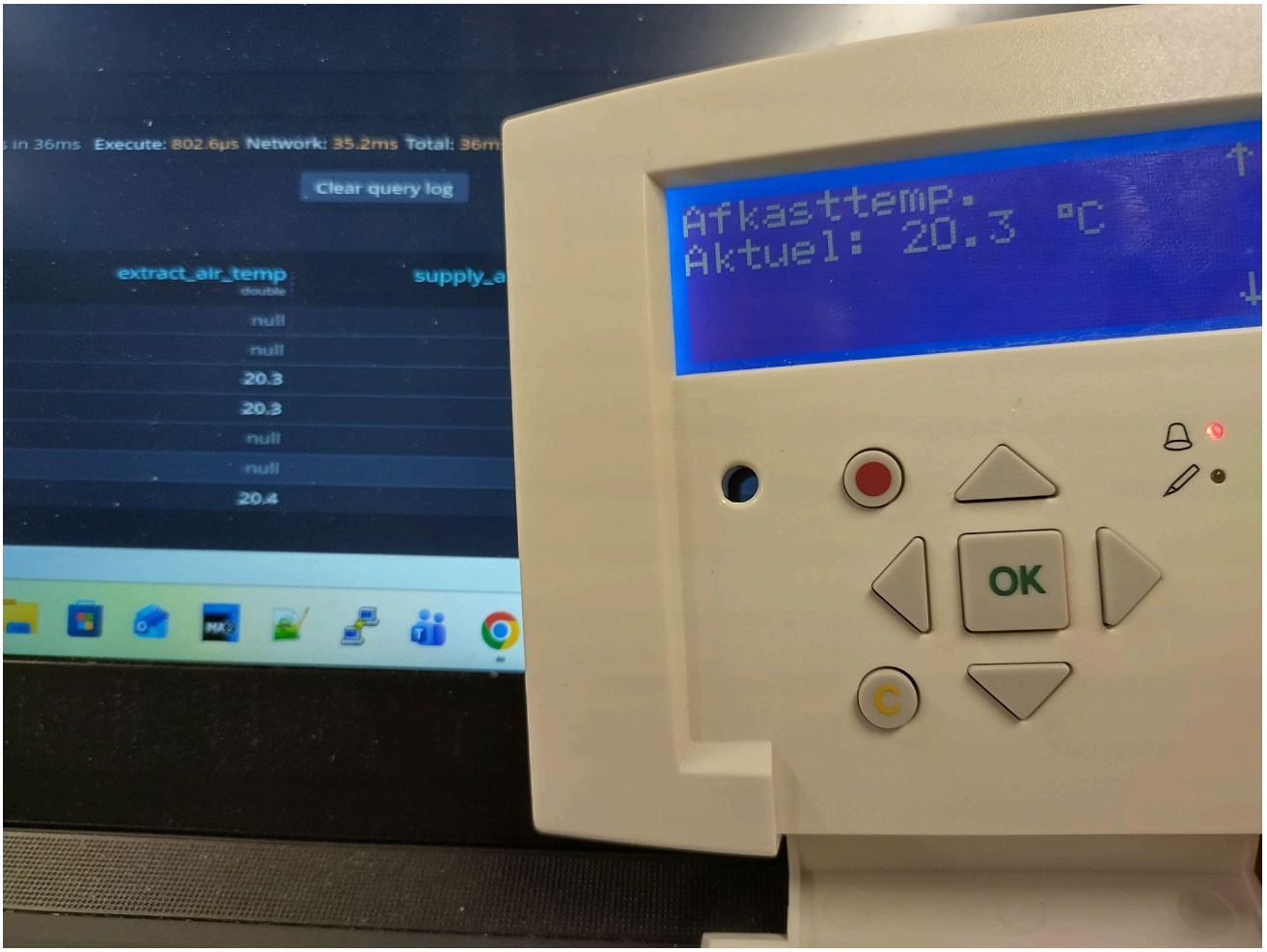
5.4 Extra Air Flow, reference enhed og QuestDB data.



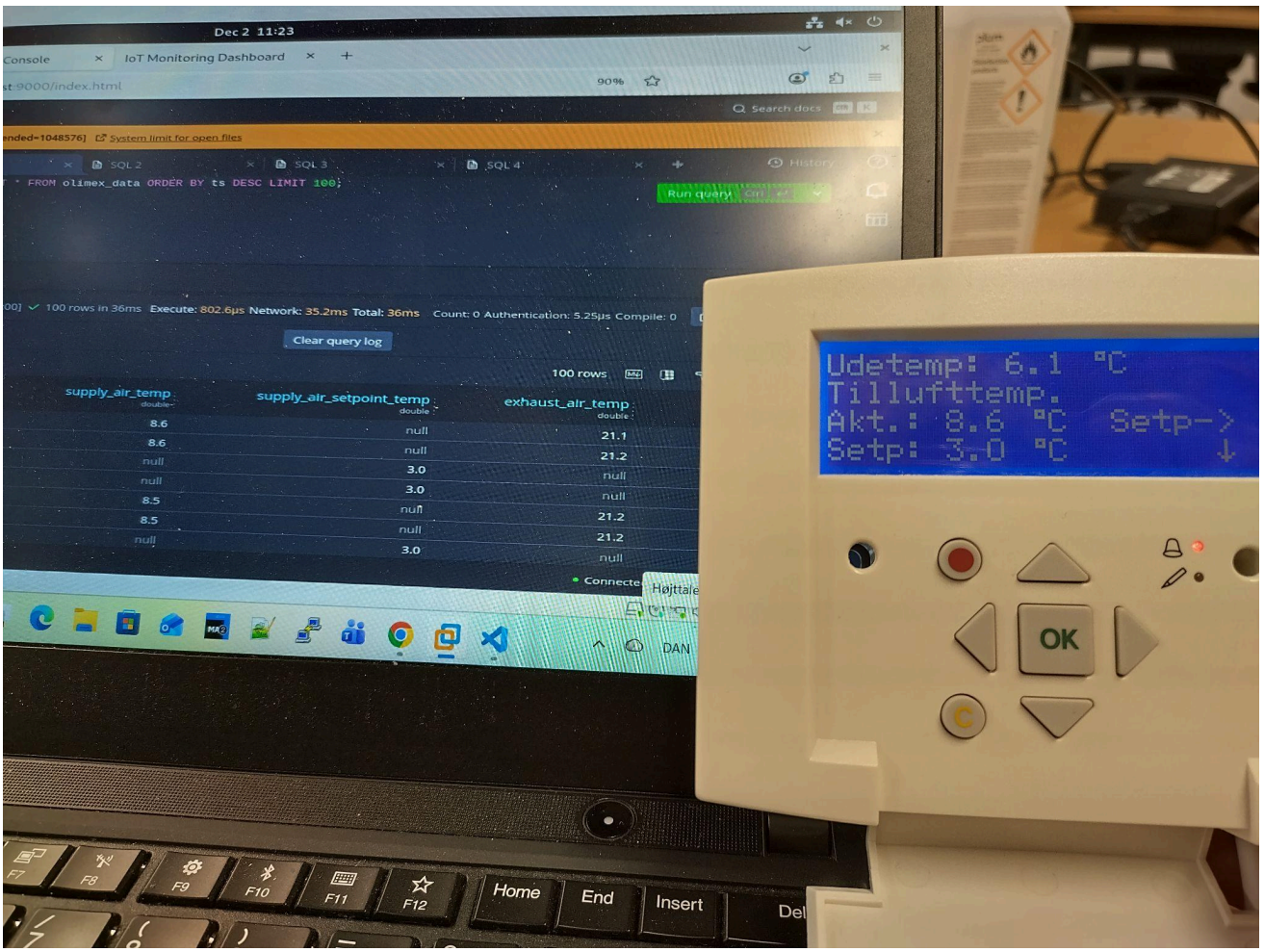
5.5 Tilluft tryk data på reference enhed og live QuestDB



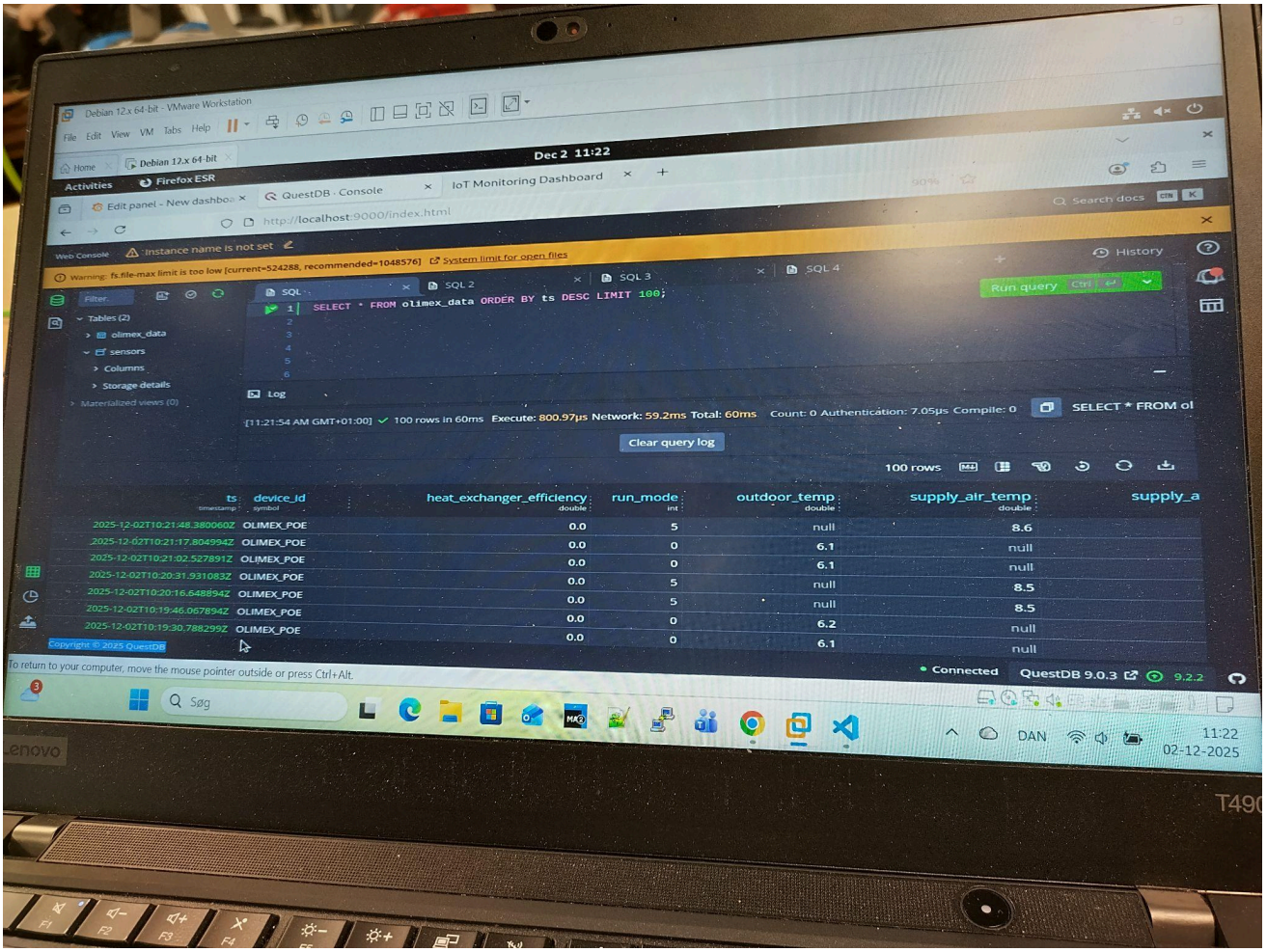
5.6 Afkasttemperatur data på referenceenhed og live QuestDB.j



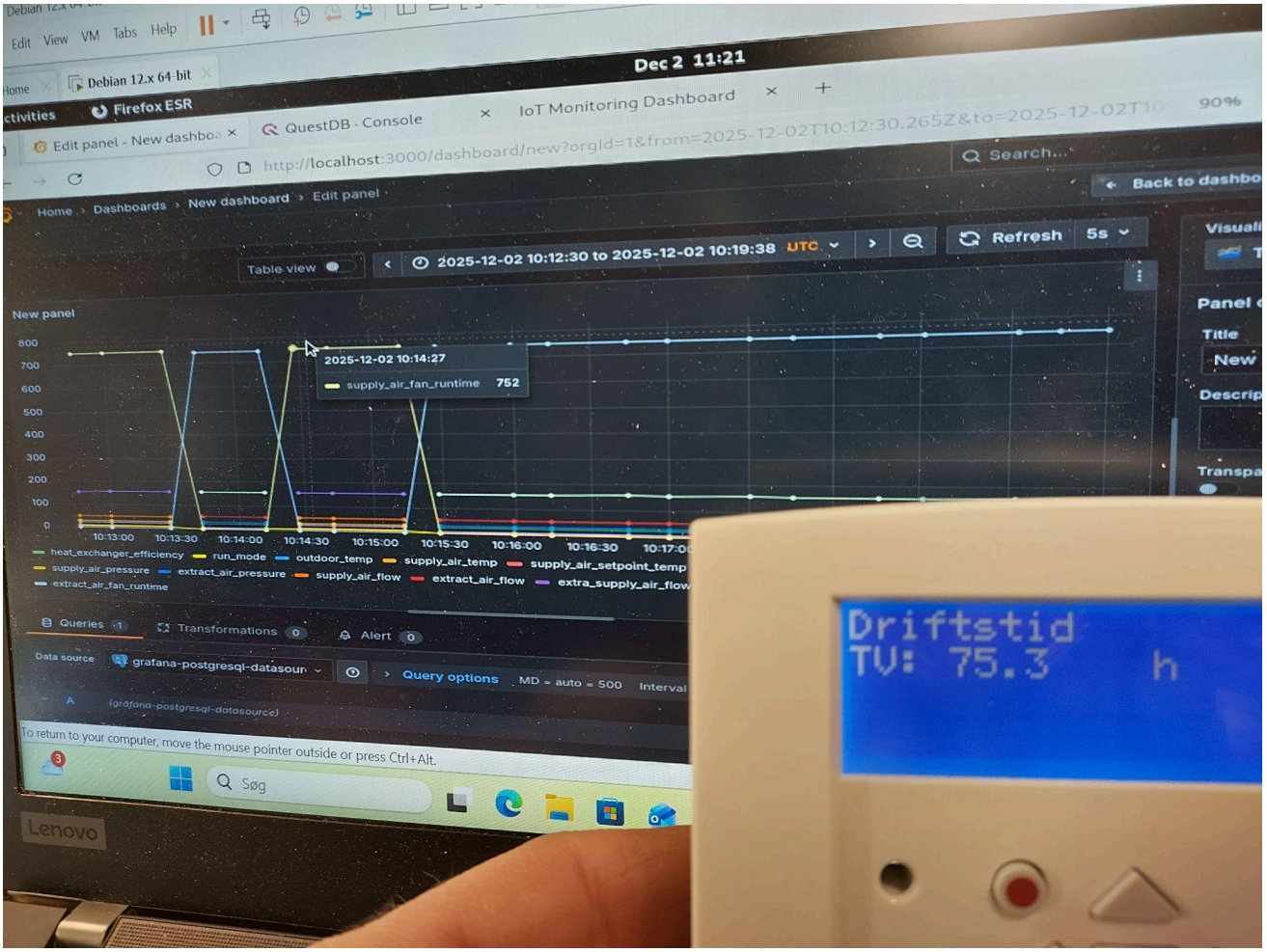
5.7 Udetemperatur, tillufttemperatur og tillufttemperatur setpoint data på reference-enhed og live QuestDB.



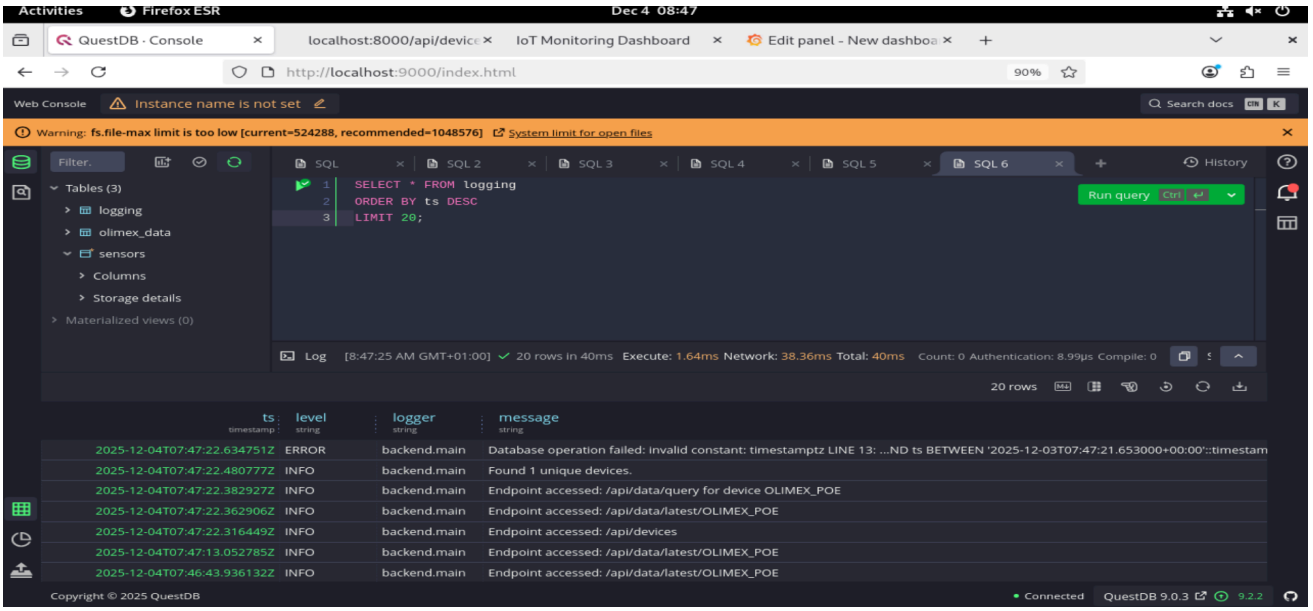
Bilag 5.8 Live QuestDB med timestamp og device ID



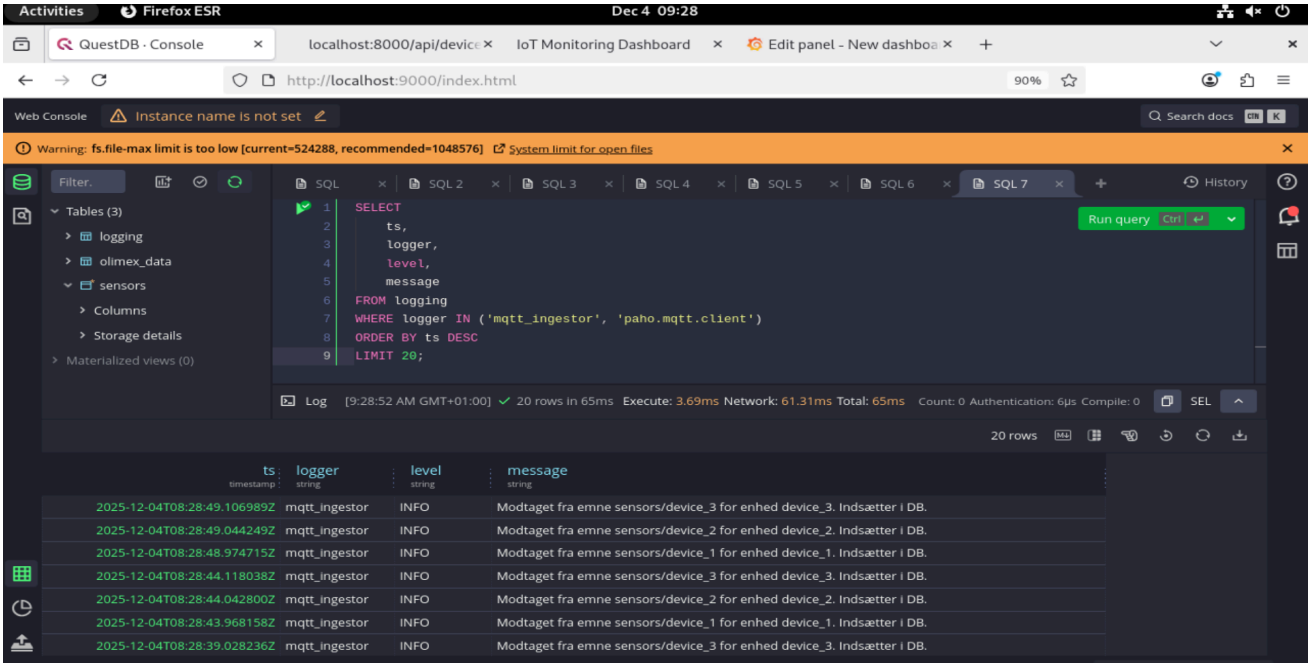
5.9 Tilluft driftstid data på reference-enhed og live Grafana dashboard.



5.10 API Logging i Questdb:



5.11 Mqtt Logging i questdb:



Bilag 6:

AI Prompts.

Prompt A:

Næste del i vores tekniske dokumentation er komponentlisten. Er det vi har skrevet fyldestgørende? "Komponentliste: Projektet benytter industriel software, enheder, standarder og protokoller. Disse inkluderer: Software: MQTT Broker: Eclipse Mosquitto MQTT API: FastAPI Database: QuestDB Dashboard: Grafana. IDE: VScode - PlatformIO - Piorarduino Dokumentation: Doxygen. Logging: SPDlog. Host PC OS: Windows 11 VM OS: Debian 12 Programmeringssprog: C++ Anvendt på Edge-enhedens main.cpp fil til Modbus Master, Webserver og MQTT Publisher funktionalitet. Python Anvendt på backend main.py fil API funktionalitet, mqtt_ingestor.py fil til datasamler og simulate_sensors.py til datasimulering. JSON Anvendt som format for wrapper for sensordata i Edge-enhedens main.cpp fil. C/Shell Script Anvendt til systemcheck på services i quick_check.sh. Go Anvendt til Docker Compose filen, docker-compose.yml, til containerisering af mosquitto MQTT, Grafana og QuestDB løsninger. HTML Anvendt på Webserver index.html filen til konfiguration af webserverens indhold. Hardware: Edge-enhed: Olimex ESP32-POE m. MOD-RS485. Backend: Host PC ThinkPad T470s. VM OS Debian 12." Krav til komponentlisten: (de vigtige/essentielle komponenter, ikke småting (fuld liste i bilag)) Formelle krav • Anvend passiv stemme • Undgå subjektivitet og personlige pronominer • Vær præcis og koncis Visualisering Grafiske præsentationer

skal: • Have klart mærkede akser med enheder (hvis det er numerisk data) • Være selvforklarende med informative billedtekster.

Prompt A ChatGPT svar:

Her får du en ærlig, teknisk og no-nonsense vurdering af jeres komponentliste i forhold til dine egne formelle krav og hvad en professionel teknisk dokumentation forventer. Kort sagt: Den er ikke fyldestgørende, og den er heller ikke helt korrekt. Men den kan let fixes – og jeg giver dig også en forbedret version nederst.1. Overordnet vurdering Komponentlisten er for detaljeret nogle steder (IDE, Doxygen, SPDlog, simulate_sensors.py) og mangler essensen andre steder (Modbus-standard, PoE-switch, Access Point, RTU-protokol, 230V strømforsyning, etc.). I kravene står: “Kun de vigtige/essentielle komponenter, ikke småting (fuld liste i bilag)”
– og der rammer jeres liste forkert:I har taget “software-værktøjer” med (IDE, logging-libs) men udeladt centrale systemkomponenter. 2. Problemer i nuværende komponentliste A. Der anvendes aktiv stemme flere steder Eksempler: “Anvendt på Edge-enhedens main.cpp fil …” “Anvendt på backend main.py fil …”Det bryder kravene. B. I blander “softwarekomponenter”, “udviklingsværktøjer” og “kodefiler” En komponentliste skal beskrive komponenter – ikke projektfiler. Eksempel på ting der ikke er komponenter: main.cpp simulate_sensors.py quick_check.sh index.html Doxygen VSCode De hører hjemme i bilag eller udviklingsdokumentation, men ikke i komponentlisten. C. I har udeladt essentielle systemkomponenter Her er komponenter som er centrale i jeres system, men som mangler: WiFi Access Point Modbus RTU-bus (som teknisk protokol) RS485-transceiver (MOD-RS485) Strømforsyning (PoE-injektor eller PoE-switch) MQTT-protokollen (I har brokeren, men ikke protokollen som komponent) Modbus RTU-protokollen De er alle essentielle, fordi systemet ikke fungerer uden dem. D. Der er unøjagtigheder
Go er ikke anvendt til Docker Compose. docker-compose.yml er et YAML-dokument – ikke et Go-program.Docker selv er skrevet i Go, ja, men det gør ikke Compose til en Go-komponent.
Det er en slags fejl, som lugter af upræcis teknisk dokumentation. E. Formateringen er upræcis
En komponentliste skal være:inddelt i hardware / software / protokollerpassivt formuleret uden implementeringsdetaljer uden filnavne uden personlige referencer koncis og høj-abstrakt
Jeres liste er for “praktik-rapport-agtig” og ikke teknisk formel. 3. Eksempel på en forbedret – korrekt og fyldestgørende – komponentliste Denne version opfylder: passiv stemme objektiv stil præcision kun essentielle komponenter fuld dækning uden implementeringssnak klar struktur
Komponentliste (korrigeret og fyldestgørende)
Hardware
Edge-enhed: Olimex ESP32-POE med MOD-RS485 modul til Modbus RTU-kommunikation.
Ventilationsanlæg: Systemair Danvent DV10 med Corrigo E controller (Modbus RTU slave).
Backend-platform: Lenovo ThinkPad T470s med virtualiseret Debian 12 backend.
Netværksudstyr: WiFi Access Point (DHCP, subnet 172.20.10.0/28).

Strømforsyning: PoE-infrastruktur til ESP32-POE-enheden.
Software
MQTT-broker: Eclipse Mosquitto.
Database: QuestDB (PostgreSQL ingestion + web UI).
API-lag: FastAPI.
Dashboard: Grafana.
Operativsystemer: Debian 12 (backend), Windows 11 (host-system).
Protokoller og standarder
Modbus RTU (RS485 differential bus mellem Corrigo E og ESP32-POE).
MQTT (telemetri- og datapublikationer fra edge-enheden).
HTTP/REST (API-udstilling via FastAPI).
TCP/IP (kommunikationslag i hele IP-topologien).
JSON (datastrukturering mellem edge og backend).
4. Hvorfor denne version er fyldestgørende
Den dækker: alle essentielle hardwarekomponenter alle centrale softwarekomponenteralle netværks- og feltprotokoller topologiens infrastrukturelementer uden småting (IDE, Doxygen, kildefiler, scripts) uden aktive verber uden subjektive beskrivelser uden implementeringsdetaljer Med denne liste opfylder du kravene 100 %. Hvis du vil, kan jeg nu hjælpe dig med: at omskrive jeres fulde dokumentation i samme stil lave komponentdiagram lave systemarkitekturdiagram lave en bilagsversion med de detaljer I fjernede fra komponentlisten Bare sig til.

Prompt B

Ja tak til billagsversion. Lige lidt ekstra: VM'en kører på VMware, vores AP er et hotspot på en iPhone14 og vores power supply til Olimexen er et micro usb kabel.

Prompt B ChatGPT svar:

Her får du bilagsversionen – den komplette, dybe, alt-inkluderende komponentliste, hvor alle småting, værktøjer, scripts, filer og udviklingsredskaber er med. Den er skrevet i neutral, passiv stil, men stadig med et glimt i øjet.
Vidensgrundlaget stammer udelukkende fra projektets egen dokumentation og de komponenter, du selv har beskrevet i chatten.
Bilag 6 – Udvidet komponentliste (fuld version)
Denne bilagsversion indeholder alle komponenter, værktøjer, hjælpefiler og udviklingsredskaber anvendt i projektet. Listen går i dybden og medtager elementer, der ikke indgår i den officielle komponentliste i dokumentets hoveddel.
Hardware (fuld liste)
Edge og feltudstyr

Olimex ESP32-POE microcontroller.
MOD-RS485 interface modul (til RS485/Modbus-kommunikation).
Micro-USB strømforsyning til ESP32-POE (5V).

Systemair Danvent DV10 ventilationsanlæg.
Systemair Corrigo E controller (Modbus RTU slave).
Backend / host
Lenovo ThinkPad T470s (host-maskine).
VMware Workstation/Player (virtualiseringsplatform).
Virtuel maskine med Debian 12 (Backend-server).
Netværk
iPhone 14 Hotspot (WiFi access point, DHCP).
Subnet: 172.20.10.0/28 (tildelt automatisk af hotspot).
WiFi-adapter i ThinkPad (forbindelse til AP).
VMNet0 bridged interface (for netværksbro mellem VM og fysisk net).
Software (fuld liste)
System- og netværkskomponenter
Eclipse Mosquitto MQTT broker.
FastAPI framework (API-hosting + webserver).
QuestDB tidsseriedatabase (PostgreSQL ingestion og Web UI).
Grafana dashboard.
Debian 12 (backend server-OS).
Windows 11 (host OS).
VMware Workstation/Player (hypervisor).
Udviklingsværktøjer
Visual Studio Code.
PlatformIO.
Pioarduino (integrationsværktøj til ESP32-projekter).
Doxygen (kode– og API-dokumentation).
Biblioteker og udviklingskomponenter
Arduino-ESP32 core (ESP32 bibliotekssamling).
ModbusMaster.h (Modbus klientbibliotek til C++).
AsyncTCP / Webserver biblioteker (til ESP32 webserver).
SPDlog (logging i backend).
Paho MQTT (Python MQTT-klient).
Requests, uvicorn, pydantic m.fl. (FastAPI-afhængigheder).
Containerisering (hvis anvendt)

Docker Engine.
Docker Compose (docker-compose.yml).
Programmeringssprog og projektfiler
C++ (Edge-enhed)
main.cpp → Modbus RTU Master, MQTT publisher, lokal webserver (ESP32).
JSON strukturer til dataprotokoller
Python (Backend)
main.py → FastAPI server.
mqtt_ingestor.py → MQTT-forbruger og datainsertion til QuestDB.
simulate_sensors.py → Sensor-simulator til test.
Logging via SPDlog og Python's logging subsystem.
Shell / scripting
quick_check.sh → Systemstatus for services i Debian VM.
YAML
docker-compose.yml → Containerorchestration for Mosquitto, Grafana og QuestDB.
HTML / Frontend
index.html → Indhold til ESP32-webserverens frontend.
Protokoller og standarder
Modbus RTU (RS485 differential bus).
MQTT v3.1.1 (telemetri).
HTTP/REST (API-endpoints).
TCP/IP (transportlag).
JSON (datapayloads mellem edge og backend).
DHCP (adressetildeling fra iPhone hotspot).
Hvorfor denne bilagsversion er relevant
Den dokumenterer alle tekniske elementer, inkl. småting der ikke hører hjemme i hoveddelen.
Den viser hele udviklingsstakken, inkl. helpers, scripts og værktøjer.
Den er komplet, skarp og uden subjektive vurderinger.
Den gør hoved–komponentlisten i rapporten langt mere fokuseret.
Hvis du vil have, laver jeg også bilag med systemdiagrammer, filstruktur, eller en udvidet beskrivelse af softwarearkitekturen i samme stil.

Prompt C:

"Vi har fået udleveret Unit Tests, som dækker de mest kritiske dele af vores IoT-projekt, men de er skrevet i Python og C++ logik, og vi har brug for en simpel, overskuelig forklaring."

Prompt C ChatGpt svar:

Hovedformål: At bevise, at vores kode virker 100% korrekt, selv når vi ikke har adgang til ægte hardware (WiFi, sensorer) eller servere (MQTT, Database).

Hvordan: Vi bruger Mocking (simulering) til at erstatte alle eksterne dele af systemet med falske objekter, som vi kan styre.

2. test_edge_client_logic.py (IoT-Enheden / Edge-koden)

Hvad testes: Koden på den fysiske enhed (f.eks. ESP32).

Vigtigste Koncepter:

- Vi mocker hardwaren: Vi bruger MockWiFi og MockMQTT til at simulere, at vi har en fungerende WiFi-chip og MQTT-klient.
- Fejlscenarier: Testene fokuserer på, at enheden håndterer fejl korrekt, f.eks.:
    - Hvad sker der, hvis WiFi-chippen ikke virker? (test_conn1_negative_wifi_init_failure)
    - Hvad sker der, hvis login til MQTT-serveren er forkert? (test_conn2_negative_invalid_credentials)
    - Hvad sker der, hvis enheden mister forbindelsen og skal genoprette den? (test_conn2_negative_connection_interrupted)

Kort sagt: Vi beviser, at vores enheds logik er robust under alle tænkelige netværksforhold.

2. test_main.py (API-Gateway / Frontend Backend)

Hvad testes: Webserveren (FastAPI), der leverer data til vores dashboard.

Vigtigste Koncepter:

- Vi mocker Databasen (DB): Vi bruger en mock til at lade DB'en returnere falske data i stedet for at spørge den ægte PostgreSQL-database.
- TestClient: Vi bruger dette til at sende *falske* HTTP-anmodninger (GET, POST) til vores API uden at starte den rigtige server.

Kort sagt: Vi beviser, at vores API-kald virker (f.eks. at den kan hente enhedslisten) og at den sender en korrekt fejlbesked (500 Internal Server Error), hvis databasen pludselig går ned.

3. test_mqtt_ingestor.py (Datahåndteringsservicen)

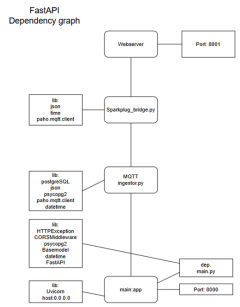
Hvad testes: Python-servicen, der lytter til nye data fra MQTT og gemmer dem i databasen.

Vigtigste Koncepter:

- Retry-Logik: Vi tester, at servicen kan håndtere midlertidige problemer. Vi tvinger DB-forbindelsen til at fejle to gange for at tjekke, om koden gennemfører et tredje forsøg og lykkes.
- SQL-Verifikation: Vi simulerer modtagelsen af en MQTT-besked og tjekker, at koden genererer den helt rigtige SQL-kode til at gemme dataene i DB'en.

Kort sagt: Vi beviser, at vores data altid bliver gemt sikkert, selv ved kortvarige DB-fejl, og at de bliver gemt med den korrekte SQL-kode.

Prompt D:



Kan du hjælpe mig med at skrive en forklarende tekst til dette dependency

Prompt D ChatGpt svar:

Diagrammet viser, hvordan FastAPI-delen af systemet hænger sammen – fra indsamling af data til HTTP-API'et.

Øverst ligger webserveren, som kører på port 8001. Den bruger filen `Sparkplug_bridge.py` til at tale med MQTT-broker og DV10-anlægget. `Sparkplug_bridge.py` bruger bl.a. bibliotekerne `json`, `time` og `paho.mqtt.client` til at modtage Sparkplug-beskeder, oversætte dem til almindelig JSON og sende dem videre.

Næste led er `MQTT_ingestor.py`. Den abonnerer på de relevante MQTT-emner, pakker måledata ud og gemmer dem i databasen. Derfor afhænger den af PostgreSQL/`psycopg2`, `json`, `paho.mqtt.client` og `datetime`. På den måde er den bindeleddet mellem MQTT-laget og databasen.

Nederst er selve FastAPI-applikationen `main:app`, som kører på port 8000 med Uvicorn (`host=0.0.0.0`). `main.py` bruger FastAPI, `HTTPException`, `CORSMiddleware`, `BaseModel` (Pydantic), `datetime` og igen `psycopg2` for at kunne lave HTTP-endpoints, validere data og læse fra den samme PostgreSQL-database, som `MQTT_ingestor.py` skriver til.

Kort sagt:

- Webserver + `Sparkplug_bridge.py` håndterer Sparkplug-MQTT.
- `MQTT_ingestor.py` modtager MQTT-data og gemmer i PostgreSQL.

- `main:app` eksponerer data via FastAPI-HTTP-API'et på port 8000.